

Blueprint de Reconstrucción

Especificación técnica COMPLETA del sistema de facturación electrónica (DGII / e-CF) de DermaLand. Contiene todo lo necesario para reconstruir el sistema desde cero con la misma funcionalidad y una interfaz equivalente: stack, setup, modelo de datos (DB), seguridad, API, motor e-CF, API UI y design system basados en negocio y guía de reconstrucción.

Next.js 15 / React 19

Supabase / Postgres

TypeScript 5.7

Tailwind 4

e-CF 31/32/33/34

XMLDSig RSA-SHA256

RLS multi-tenant

7 roles / 18 permisos

Documento	Blueprint de reconstrucción - sistema de facturación (DGII / e-CF)
Producto	DermaLand - Plataforma SaaS de facturación electrónica
Version doc.	1.0
Fecha	2026-06-12
Nivel	Reconstrucción (build-from-scratch)
Estado fiscal	Fases 0-12 listas. Fase G / producción: BLOQUEADAS por política

Indice

0. Como usar este documento	3	8.3 Flujo POS -> e-CF	60
1. Arquitectura y stack	4	8.4 Cierre de caja con porcentaje	60
1.1 Vision general	4	8.5 Maquinas de estado	61
1.2 Stack tecnologico	4	9. Fases, gaps y guia de reconstruccion	62
1.3 Estructura del monorepo	4	9.1 Estado por fase	62
2. Setup y despliegue	6	9.2 Gaps conocidos (para completar la...	62
2.1 Requisitos	6	9.3 Orden de construccion recomendado	62
2.2 Scripts	6		
2.3 Levantar local desde cero	6		
2.4 Variables de entorno	6		
2.5 Configuracion Next.js / Vercel	7		
2.6 Testing y CI	7		
3. Modelo de datos (DDL completo)	8		
ESQUEMA DE BASE DE DATOS COMPLETO -...	8		
TABLAS POR DOMINIO	8		
RESUMEN DE FUNCIONES PL/pgSQL	47		
ENUMERACIONES SINTETIZADAS	47		
NOTAS FINALES	48		
4. Seguridad, RLS, roles y autenticacion	50		
4.1 Funciones SQL de auth	50		
4.2 Patron RLS	50		
4.3 Roles (7) y permisos (18)	50		
4.4 JWT, sesion y clientes Supabase	51		
5. Motor de comprobante fiscal (e-CF)	52		
5.1 Estructura del XML por tipo (orden exacto del...	52		
5.2 Tipo de entrada (TypeScript)	52		
5.3 Firma XMLDSig (signer.ts)	53		
5.4 Validacion XSD (validator.ts)	53		
5.5Codigo de seguridad y QR	53		
5.6 PDF (pdf.ts)	53		
5.7 Mapeos y reglas fiscales	54		
5.8 Dry-run y estado del envio	54		
6. API y contratos	55		
6.1 Routing (App Router)	55		
6.2 Route handlers (endpoints)	55		
6.3 Ejemplos de contrato	55		
6.4 Server actions	56		
6.5 Killswitches / feature flags	56		
7. UI y design system	57		
7.1 Shell de navegacion	57		
7.2 Design system (tokens)	57		
7.3 Componentes UI base (custom, en...	57		
7.4 Pantallas del modulo DGII (prioridad)	58		
7.5 Pantallas POS / Caja	59		
8. Logica de negocio y estados	60		
8.1 ITBIS y totales	60		
8.2 Secuencias e-NCF (reserva atomica)	60		

0. Como usar este documento

Este blueprint esta organizado en partes que, en conjunto, permiten reconstruir el sistema:

- Parte 1 - Arquitectura y stack: vision general, monorepo y tecnologias.
- Parte 2 - Setup y despliegue: como levantar el proyecto desde cero (local + Vercel).
- Parte 3 - Modelo de datos: DDL completo de todas las tablas, constraints, indices y seeds.
- Parte 4 - Seguridad: funciones SQL de auth, RLS por tabla, roles, permisos y JWT.
- Parte 5 - Motor e-CF: estructura XML por tipo, firma, validacion, QR, codigo de seguridad, PDF y mapeos.
- Parte 6 - API y contratos: endpoints, server actions, errores y killswitches.
- Parte 7 - UI y design system: shell de navegacion, pantallas, componentes y tokens de marca.
- Parte 8 - Logica de negocio: POS, cierre de caja, secuencias y maquinas de estado.
- Parte 9 - Fases, gaps y guia de reconstruccion: orden de construccion recomendado.

> Politica operativa: la emision fiscal real frente a la DGII (Fase G / testecf y produccion > ambiente "ecf") esta BLOQUEADA y requiere autorizacion explicita por turno ademas de la > postulacion del contribuyente ante la DGII. Reconstruir el sistema NO implica habilitarla.

1. Arquitectura y stack

1.1 Vision general

DermaLand es una aplicacion SaaS multi-tenant (un negocio = un business) construida como un monorepo pnpm. La app web es Next.js 15 (App Router) con React 19 y TypeScript. La persistencia es Supabase (Postgres) accedida por SQL directo y un patron de repositorios; no hay ORM tradicional (Prisma/Drizzle). El aislamiento entre negocios se garantiza con Row Level Security (RLS) por business_id. El modulo fiscal (DGII / e-CF) cubre todo el ciclo del Comprobante Fiscal Electronico de Republica Dominicana.

Hay un interruptor global DATA_SOURCE que alterna entre datos mock en memoria (modo demo) y supabase (backend real). Esto permite levantar la app y navegar todas las pantallas sin backend.

1.2 Stack tecnologico

Capa	Tecnologia	Version
Framework	Next.js (App Router)	15.5.18
UI runtime	React	19.0.0
Lenguaje	TypeScript (strict, noUncheckedIndexedAccess)	5.7.3
Estilos	Tailwind CSS + @tailwindcss/postcss	4.1.0
Primitivos UI	Custom (NO shadcn/Radix)	-
Iconos	lucide-react	1.14.0
Validacion	Zod	4.4.3
Estado cliente	Zustand + clsx + tailwind-merge	-
Persistencia	Supabase JS + SSR	2.105.3 / 0.10.2
Offline	idb (IndexedDB)	8.0.3
PDF	pdfkit	0.18.0
QR	qrcode + @zxing/browser	1.5.4 / 0.2.0
XML	xmlbuilder2 / xml-crypto / xmllint-wasm	4.0.3 / 6.1.2 / 5.2.0
Cripto	node-forge	1.3.1
Tests	Vitest + Playwright	4.1.x / 1.59.1
Gestor paquetes	pnpm	10.33.0

1.3 Estructura del monorepo

```
dermaland/  
  apps/web/           # Next.js 15 (la app)  
    src/  
      app/            # App Router  
        (app)/         # shell de negocio (requiere auth)  
        (super-admin)/ # admin de plataforma  
        api/           # route handlers  
        login/         # login  
      components/  
        layout/        # AppShell, Sidebar, Header
```

```

    ui/                # Button, Card, Badge, Input, Table, Tabs, Toast...
    features/          # dgii, pos, sales, inventory, inventory-counts...
    lib/               # env.ts, supabase/, mock-data/, utils/
    server/            # auth/, repositories/ (mock|supabase|factory), services/dgii/
    types/             # modelos de dominio
    middleware.ts       # auth + gating super-admin
    next.config.ts, vitest.config.ts, playwright.config.ts, postcss.config.mjs
    supabase/migrations/ # 0001..0009 (SQL versionado)
    docs/dgii/         # plan maestro, XSD oficiales, runbooks, matriz
    package.json, pnpm-workspace.yaml, .env.example

```

El workspace declara apps/ y packages/ (este ultimo aun vacio: lugar previsto para librerias compartidas).

2. Setup y despliegue

2.1 Requisitos

- Node.js 22 LTS (la app corre tambien en Node 24).
- pnpm 10.33.0+.
- Git. Cuenta Supabase (si DATA_SOURCE=supabase). Cuenta Vercel (despliegue).

2.2 Scripts

Raiz (package.json): dev, build, start, typecheck (todos pnpm --filter web ...).

App (apps/web/package.json):

Comando	Accion
next dev -p 3031	Dev server en http://localhost:3031 (hot reload)
next build	Build de produccion
next start -p 3031	Servidor de produccion
tsc --noEmit	Typecheck
vitest run / vitest	Unit tests (una vez / watch)
playwright test	E2E (requiere dev server)
playwright install --with-deps chromium	Instala navegador para E2E

2.3 Levantar local desde cero

- 1. git clone <repo> dermaland && cd dermaland
- 2. cp .env.example .env y completar valores (ver 2.4). Para arranque rapido: DATA_SOURCE=mock.
- 3. pnpm install
- 4. pnpm dev -> http://localhost:3031
- 5. (Opcional, backend real) configurar Supabase: aplicar migraciones supabase/migrations/0001..0009 (en orden), correr seeds, generar tipos (supabase gen types typescript) y poner DATA_SOURCE=supabase.
- 6. Validar: GET /api/health debe responder 200.

2.4 Variables de entorno

Fuente: .env.example; validacion con Zod en src/lib/env.ts (permisivo en dev, estricto en prod).

Variable	Requerida	Para que
NODE_ENV	siempre	development / test / production
NEXT_PUBLIC_APP_URL / APP_URL	siempre	URL base (http://localhost:3031 dev)
DATA_SOURCE	siempre	"mock" o "supabase" (killswitch #1)
NEXT_PUBLIC_SUPABASE_URL	si supabase	URL del proyecto
NEXT_PUBLIC_SUPABASE_ANON_KEY	si supabase	anon key (RLS es el limite de seguridad)
SUPABASE_SERVICE_ROLE_KEY	si supabase	bypass RLS - solo server, nunca commitear

Variable	Requerida	Para que
DATABASE_URL	si migraciones	conexion directa Postgres
JWT_SECRET	recomendado	32+ chars para cookies internas
SESSION_COOKIE_NAME	opcional	default "dermaland-session"
DGII_ENVIRONMENT	Fase 5+	testecf / certecf / ecf (legacy cert/prod)
DGII_CERTIFICATE_PATH / _PASSWORD	Fase 5+	ubicacion y clave del .p12
DGII_CERT_ENCRYPTION_KEY	Fase F	32+ chars para cifrar el blob del certificado
DGII_BASE_URL_TESTECF/_CERTECF/_ECF	opcional	override de URLs DGII
DGII_TESTECF_SEND_ENABLED	opcional	killswitch envio real (default false)
WHATSAPP_* / OPENAI_* / RESEND_* / SENTRY_* / UPSTASH_*	por modulo	integraciones opcionales

2.5 Configuracion Next.js / Vercel

```
// next.config.ts
const nextConfig = {
  reactStrictMode: true,
  serverExternalPackages: ["pdfkit", "xmllint-wasm", "node-forge", "xml-crypto"],
  outputFileTracingIncludes: {
    "/api/dgii/certificate/**": ["../../docs/dgii/xsd/*.xsd"],
  },
};
```

- Root directory en Vercel: apps/web. Framework: Next.js. Produccion: <https://dermaland.vercel.app>.
- serverExternalPackages evita bundlear paquetes con binarios/wasm/fs.
- outputFileTracingIncludes embebe los XSD oficiales en la lambda de validacion.

2.6 Testing y CI

- Vitest (entorno node, alias @->src, stub de server-only): src/*.test.ts + tests/unit/.
- Playwright (Chromium + mobile Pixel 7, baseUrl :3031, retries 2 en CI).
- GitHub Actions (.github/workflows/ci.yml): job validate (typecheck + unit + build) y job e2e

(con continue-on-error por ahora). Variables CI: DATA_SOURCE=mock.

3. Modelo de datos (DDL completo)

Esta parte transcribe el esquema completo extraído de las migraciones supabase/migrations/0001..0009: todas las tablas por dominio, sus columnas (tipo, default, not null), constraints (PK/FK/UNIQUE/CHECK), índices, enums y seeds. Todas las tablas de negocio incluyen business_id y están protegidas por RLS (ver Parte 4).

ESQUEMA DE BASE DE DATOS COMPLETO - DermaLand

ORDEN DE APLICACIÓN DE MIGRACIONES

#	Archivo	Introducción
0001	0001_phase1_core.sql	Tablas core: tenancia (businesses, branches, warehouses), usuarios, audit_logs, planes. RLS y funciones auth.
0002a	0002a_clients.sql	Tabla clients (CRM/clientes), referencias de proformas/e-CF.
0002	0002_phase2_inventory.sql	Inventario: productos, marcas, laboratorios, lotes, movimientos, conteos físicos, cuarentena, recalls.
0003	0003_dgii_pos.sql	DGII (e-CF): configuración, certificados, secuencias, POS (cajas, proformas, e-CF, cierres), recepción de facturas.
0004	0004_dgii_permissions_seed.sql	Tabla permissions con 18 permisos DGII/cash (catálogo).
0005	0005_dgii_role_permissions_seed.sql	Tablas roles y role_permissions, mapeo N:M de 7 roles con 18 permisos.
0006	0006_auth_helpers_jwt_metadata.sql	Actualiza funciones auth_business_id() y auth_is_platform_admin() para leer JWT metadata anidado.
0007	0007_audit_logs_insert_policy.sql	Añade policy INSERT a audit_logs con guardia business_id y user_id.
0008	0008_security_advisor_fixes.sql	Optimizaciones RLS (InitPlan, security_invoker, search_path). Sin cambios de lógica.
0009	0009_rls_initplan_remaining.sql	Envuelve auth_*(*) en (select ...) en todas las políticas restantes.

TABLAS POR DOMINIO

A. PLATAFORMA Y TENANCY

platform_users

Propósito: Usuarios de la plataforma (super admin).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
email	text	-	[x]	Único por usuario admin
full_name	text	-	[x]	Nombre completo
is_active	boolean	true	[x]	Usuario activo
two_factor_enabled	boolean	false	[x]	2FA habilitado
created_at	timestampz	now()	[x]	Timestamp de creación

Columna	Tipo	DEFAULT	NOT NULL	Comentario
updated_at	timestampz	now()	[x]	Timestamp de última actualización

Constraints:

- PRIMARY KEY: id
- UNIQUE: email
- RLS: platform_users_select (SELECT solo si auth_is_platform_admin())

Índices:

- (Ninguno adicional más allá de PK/UNIQUE)

plans

Propósito: Catálogo de planes de suscripción (global).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
name	text	-	[x]	Nombre único del plan (p.ej. 'Básico', 'Pro', 'Business / POS')
monthly_price_usd	numeric(10,2)	-	[x]	Precio mensual en USD
features	jsonb	'[]'	[x]	Array de características (JSON)
limits	jsonb	'{}'	[x]	Límites por plan (JSON)
highlight	boolean	false	[x]	Destaca el plan en UI
created_at	timestampz	now()	[x]	Timestamp de creación

Constraints:

- PRIMARY KEY: id
- UNIQUE: name
- RLS: plans_select (SELECT: true), plans_admin_insert, plans_admin_update, plans_admin_delete (solo auth_is_platform_admin())

Seed (valores iniciales):

```
('00000000-0000-0000-0000-000000000001', 'Básico', 39, '[]'::jsonb, '{}'::jsonb, false)
('00000000-0000-0000-0000-000000000002', 'Pro', 99, '[]'::jsonb, '{}'::jsonb, false)
('00000000-0000-0000-0000-000000000003', 'Business / POS', 199, '[]'::jsonb, '{}'::jsonb, true)
('00000000-0000-0000-0000-000000000004', 'Premium IA', 349, '[]'::jsonb, '{}'::jsonb, false)
('00000000-0000-0000-0000-000000000005', 'Enterprise', 0, '[]'::jsonb, '{}'::jsonb, false)
```

businesses

Propósito: Tenant/empresa (multi-tenant core).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
legal_name	text	-	[x]	Razón social

Columna	Tipo	DEFAULT	NOT NULL	Comentario
commercial_name	text	-	[x]	Nombre comercial
rnc	text	-	[x]	RNC/NIT del negocio (Rep. Dominicana)
country	text	'República Dominicana'	[x]	País
phone	text	-	-	Teléfono
whatsapp	text	-	-	WhatsApp
email	text	-	-	Email contacto
instagram_url	text	-	-	URL Instagram
logo_url	text	-	-	URL logo
dgii_enabled	boolean	false	[x]	DGII habilitado (control gating)
plan_id	uuid	-	[x]	FK -> plans(id)
status	text	'trial'	[x]	CHECK: 'active'
created_at	timestamp z	now()	[x]	Timestamp de creación
updated_at	timestamp z	now()	[x]	Timestamp de última actualización
deleted_at	timestamp z	-	-	Soft delete

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: plan_id -> plans(id)
- CHECK: status in ('active','suspended','trial','past_due')
- RLS: businesses_sel (SELECT: admin OR id=biz), businesses_ins (INSERT: admin), businesses_upd (UPDATE: admin OR id=biz), businesses_del (DELETE: admin)

Índices:

- businesses_status_idx: (status)

Seed:

```
('00000000-0000-0000-0000-00000000d001', 'DermaLand SRL', 'DermaLand', '1-32-59077-5', 'República Dominicana', '+1 809-226-5252', '+1 809-226-5252', 'dermalandrd@gmail.com', '00000000-0000-0000-0000-000000000003', 'trial')
```

branches

Propósito: Sucursales de un business.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
code	text	-	[x]	Código único por business (ej. 'STG-01')
name	text	-	[x]	Nombre sucursal
address	text	-	[x]	Dirección

Columna	Tipo	DEFAULT	NOT NULL	Comentario
city	text	-	[x]	Ciudad
province	text	-	[x]	Provincia/estado
country	text	-	[x]	País
phone	text	-	-	Teléfono
whatsapp	text	-	-	WhatsApp
email	text	-	-	Email
is_pilot	boolean	false	[x]	Sucursal piloto (para pruebas DGII)
show_on_website	boolean	true	[x]	Mostrar en web
status	text	'active'	[x]	CHECK: 'active'\
created_at	timestamp z	now()	[x]	Timestamp de creación
updated_at	timestamp z	now()	[x]	Timestamp de última actualización
deleted_at	timestamp z	-	-	Soft delete

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, code)
- CHECK: status in ('active','inactive')
- RLS: branches_sel (SELECT: business_id=biz OR admin), branches_ins/upd/del (business_id=biz)

Índices:

- branches_business_idx: (business_id)

Seed:

```
('00000000-0000-0000-0000-00000000b001', '00000000-0000-0000-0000-00000000d001', 'STG-01',
'DermaLand Santiago', 'Calle E. León Jiménez No. 47, Esq. Mayagüez, Reparto del Este',
'Santiago de los Caballeros', 'Santiago', 'República Dominicana',
'+1 809-226-5252', NULL, 'santiago@dermaland.do', true, true, 'active')
```

warehouses

Propósito: Almacenes por sucursal.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
code	text	-	[x]	Código único por business
name	text	-	[x]	Nombre almacén
description	text	-	-	Descripción

Columna	Tipo	DEFAULT	NOT NULL	Comentario
is_main	boolean	false	[x]	Almacén principal
created_at	timestamptz	now()	[x]	Timestamp de creación
updated_at	timestamptz	now()	[x]	Timestamp de última actualización

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- UNIQUE: (business_id, code)
- RLS: warehouses_all (business_id=biz)

B. AUTH Y PERMISOS

users

Propósito: Usuarios del tenant (mapeo a auth.users de Supabase).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	-	[x]	PK: mismo id que auth.users
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
email	text	-	[x]	Email único
full_name	text	-	[x]	Nombre completo
phone	text	-	-	Teléfono
role	text	'cashier'	[x]	CHECK: 'admin\'
branch_ids	uuid[]	'{}'	[x]	Array de sucursales autorizadas
two_factor_enabled	boolean	false	[x]	2FA habilitado
status	text	'active'	[x]	CHECK: 'active\'
last_login_at	timestamptz	-	-	Última conexión
avatar_color	text	'#1A7F8E'	[x]	Color de avatar
created_at	timestamptz	now()	[x]	Timestamp de creación
updated_at	timestamptz	now()	[x]	Timestamp de última actualización
deleted_at	timestamptz	-	-	Soft delete

Enums:

- role: admin, manager, cashier, inventory, supervisor, auditor
- status: active, invited, disabled

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE

- UNIQUE: email
- CHECK: role in ('admin','manager','cashier','inventory','supervisor','auditor')
- CHECK: status in ('active','invited','disabled')
- RLS: users_sel/ins/upd/del (business_id=biz)

Índices:

- users_business_idx: (business_id)

permissions

Propósito: Catálogo de permisos granulares (creado en 0004).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
code	text	-	[x]	PK: código único (ej. 'dgii:invoices:send')
module	text	-	[x]	Módulo (ej. 'Facturas electrónicas')
description	text	-	[x]	Descripción legible
is_destructive	boolean	false	[x]	Marca operación destructiva (auditable)
created_at	timestampz	now()	[x]	Timestamp de creación

Permisos DGII (18):

Code	Module	Destructive	Descripción
dgii:configure	Configuración DGII	false	Editar RNC, ambiente, URLs
dgii:certificate:upload	Certificado	true	Subir/reemplazar .p12
dgii:sequences:manage	Secuencias	true	Importar/gestionar secuencias
dgii:invoices:generate_xml	Facturas electrónicas	false	Generar XML e-CF
dgii:invoices:validate_xml	Facturas electrónicas	false	Validar contra XSD DGII
dgii:invoices:sign	Facturas electrónicas	true	Firmar XML con certificado
dgii:invoices:send	Facturas electrónicas	true	Enviar XML a DGII
dgii:invoices:check_status	Facturas electrónicas	false	Consultar estado TrackId
dgii:invoices:download_xml	Facturas electrónicas	false	Descargar XML
dgii:invoices:download_pdf	Facturas electrónicas	false	Descargar PDF
dgii:credit_notes:create	Notas de crédito	true	Crear NC (e-CF 34)
dgii:reports:view	Reportes	false	Ver reportes fiscales
dgii:certification:run_tests	Pre-certificación	false	Pruebas en testecf
cash:open	Caja/cierre	false	Abrir sesión caja
cash:close	Caja/cierre	true	Cerrar sesión caja
cash:change_closing_percentage	Caja/cierre	true	Cambiar % e-CF en cierre
cash:authorize_below_100_percent	Caja/cierre	true	Autorizar cierres < 100%
cash:reverse_closing	Caja/cierre	true	Reversar cierre confirmado

Constraints:

- PRIMARY KEY: code
- RLS: permissions_read_all (SELECT: true, cualquier usuario)

roles

Propósito: Catálogo de roles del sistema (creado en 0005).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
code	text	-	[x]	PK: código único (ej. 'admin')
label	text	-	[x]	Etiqueta legible (ej. 'Admin')
description	text	-	[x]	Descripción
created_at	timestampz	now()	[x]	Timestamp de creación

Roles (7):

Code	Label	Descripción
super_admin	Súper Admin	Acceso total. Solo personal interno.
admin	Admin	Administra negocio: sucursales, usuarios, config.
manager	Gerente de sucursal	Gestiona sucursal asignada: POS, inventario, conteos.
cashier	Cajero	Atiende ventas POS, abre/cierra caja.
inventory	Inventario	Recepciones, conteos, ajustes.
supervisor	Supervisor	Aprueba ajustes, autoriza cierres sensibles.
auditor	Auditor	Solo lectura: auditoría, reportes, logs.

Constraints:

- PRIMARY KEY: code
- RLS: roles_read_all (SELECT: true)

role_permissions

Propósito: Matriz N:M rol <-> permiso (creado en 0005).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
role_code	text	-	[x]	FK -> roles(code) ON DELETE CASCADE
permission_code	text	-	[x]	FK -> permissions(code) ON DELETE CASCADE
granted_at	timestampz	now()	[x]	Timestamp de otorgamiento

Constraints:

- PRIMARY KEY: (role_code, permission_code)
- FOREIGN KEY: role_code -> roles(code) ON DELETE CASCADE
- FOREIGN KEY: permission_code -> permissions(code) ON DELETE CASCADE
- RLS: role_permissions_read_all (SELECT: true)

Asignación por rol:

Rol	Permisos (count)
super_admin	18 (todos)
admin	18 (todos)
manager	12 (dgii: generate_xml, validate_xml, sign, send, check_status, download_xml, download_pdf, credit_notes:create, reports:view; cash: open, close, change_closing_percentage)
cashier	4 (dgii: generate_xml, download_pdf; cash: open, close)
inventory	0
supervisor	3 (dgii: reports:view; cash: authorize_below_100_percent, reverse_closing)
auditor	4 (dgii: reports:view, invoices:check_status, invoices:download_xml, invoices:download_pdf)

audit_logs

Propósito: Auditoria de acciones por tenant.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK: identificador único
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
user_id	uuid	-	-	FK -> users(id) ON DELETE SET NULL
user_name	text	-	-	Nombre del usuario (snapshot)
action	text	-	[x]	Acción realizada (ej. 'created', 'updated', 'deleted')
entity	text	-	[x]	Entidad afectada (ej. 'proforma', 'electronic_invoice')
entity_id	text	-	[x]	ID de la entidad
branch_id	uuid	-	-	FK -> branches(id) (referencia opcional)
metadata	jsonb	-	-	Datos adicionales (antes/después)
ip_addresses	inet	-	-	IP del cliente
created_at	timestamp tz	now()	[x]	Timestamp de creación

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: user_id -> users(id) ON DELETE SET NULL
- FOREIGN KEY: branch_id -> branches(id) (sin constraint explícito listado pero referencia)
- RLS: audit_logs_select (SELECT: business_id=biz), audit_logs_insert (INSERT: business_id=biz AND (user_id IS NULL OR user_id=auth.uid()))

Índices:

- audit_logs_business_created_idx: (business_id, created_at DESC)

C. CATÁLOGO Y INVENTARIO

brands

Propósito: Marcas de productos.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[X]	PK
business_id	uuid	-	[X]	FK -> businesses(id) ON DELETE CASCADE
name	text	-	[X]	Nombre de marca
product_count	int	0	[X]	Contador de productos (desnormalizado)
created_at	timestampz	now()	[X]	Timestamp
updated_at	timestampz	now()	[X]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, name)
- RLS: brands_all (business_id=biz)

laboratories

Propósito: Laboratorios (fabricantes).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[X]	PK
business_id	uuid	-	[X]	FK -> businesses(id) ON DELETE CASCADE
name	text	-	[X]	Nombre del laboratorio
country	text	-	-	País de origen
created_at	timestampz	now()	[X]	Timestamp
updated_at	timestampz	now()	[X]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- RLS: laboratories_all (business_id=biz)

product_categories

Propósito: Categorías jerárquicas de productos.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[X]	PK
business_id	uuid	-	[X]	FK -> businesses(id) ON DELETE CASCADE
name	text	-	[X]	Nombre categoría
parent_id	uuid	-	-	FK -> product_categories(id) (autorreferencia)
description	text	-	-	Descripción
created_at	timestampz	now()	[X]	Timestamp

Columna	Tipo	DEFAULT	NOT NULL	Comentario
updated_at	timestamp z	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: parent_id -> product_categories(id) (sin ON DELETE listado; NULL para raíz)
- RLS: product_categories_all (business_id=biz)

products

Propósito: Catálogo de productos.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
sku	text	-	[x]	SKU único por business
barcode	text	-	-	Código de barras (único por business si no null)
name	text	-	[x]	Nombre producto
description	text	-	-	Descripción
brand_id	uuid	-	-	FK -> brands(id)
laboratory_id	uuid	-	-	FK -> laboratories(id)
category_id	uuid	-	-	FK -> product_categories(id)
unit	text	'unidad'	[x]	Unidad de medida
pharmaceutical_form	text	-	-	Forma farmacéutica (cápsula, jarabe, etc.)
presentation	text	-	-	Presentación (10 ml, 30 comp, etc.)
active_ingredient	text	-	-	Principio activo
concentration	text	-	-	Concentración
sanitary_registry	text	-	-	Registro sanitario
storage_temperature	text	-	-	Temperatura de almacenamiento
requires_prescription	boolean	false	[x]	Requiere receta
controlled	boolean	false	[x]	Sustancia controlada
cost	numeric(12,2)	0	[x]	Costo unitario
price	numeric(12,2)	-	[x]	Precio de venta
itbis_rate	numeric(5,2)	18	[x]	Tasa ITBIS (%)
min_stock	int	0	[x]	Stock mínimo
max_stock	int	0	[x]	Stock máximo

Columna	Tipo	DEFAULT	NOT NULL	Comentario
image_url	text	-	-	URL imagen
active	boolean	true	[x]	Producto activo
sellable	boolean	true	[x]	Vendible
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp
deleted_at	timestampz	-	-	Soft delete

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: brand_id -> brands(id)
- FOREIGN KEY: laboratory_id -> laboratories(id)
- FOREIGN KEY: category_id -> product_categories(id)
- UNIQUE: (business_id, sku)
- UNIQUE (parcial): (business_id, barcode) WHERE barcode IS NOT NULL
- RLS: products_all (business_id=biz)

Índices:

- products_barcode_unique: (business_id, barcode) WHERE barcode IS NOT NULL
- products_business_active_idx: (business_id, active)

product_lots

Propósito: Lotes/números de serie de productos (por almacén).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
product_id	uuid	-	[x]	FK -> products(id) ON DELETE CASCADE
warehouse_id	uuid	-	[x]	FK -> warehouses(id) ON DELETE CASCADE
warehouse_location_id	uuid	-	-	Ubicación física (pendiente modelo)
lot_number	text	-	[x]	Número lote/lote
manufactured_at	date	-	-	Fecha fabricación
expires_at	date	-	[x]	Fecha vencimiento
received_at	timestampz	now()	[x]	Fecha recepción
initial_quantity	int	-	[x]	Cantidad inicial
current_quantity	int	-	[x]	Cantidad actual (stock)
unit_cost	numeric(12,2)	-	[x]	Costo unitario

Columna	Tipo	DEFAULT	NOT NULL	Comentario
unit_price	numeric(12,2)	-	-	Precio unitario (sobrescribe producto)
supplier_id	uuid	-	-	Proveedor (referencia pendiente)
purchase_invoice	text	-	-	Número factura de compra
status	text	'available'	[x]	CHECK: 'available'
notes	text	-	-	Notas
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Enums:

- status: available, quarantine, expired, recalled, damaged, returned

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id) ON DELETE CASCADE
- FOREIGN KEY: warehouse_id -> warehouses(id) ON DELETE CASCADE
- UNIQUE: (business_id, product_id, lot_number, warehouse_id)
- CHECK: status in ('available','quarantine','expired','recalled','damaged','returned')
- RLS: product_lots_all (business_id=biz)

Índices:

- product_lots_fefo_idx: (business_id, product_id, expires_at) WHERE status='available' AND current_quantity > 0 (FEFO: First Expiring, First Out)
- product_lots_expiring_idx: (business_id, expires_at) WHERE status='available'

Funciones:

- select_lot_for_sale(p_business_id uuid, p_product_id uuid, p_branch_id uuid DEFAULT NULL) -> uuid: Devuelve el lote más próximo a vencer disponible (FEFO). Usado por POS y server actions.

inventory_movements

Propósito: Registro de movimientos de inventario (entrada, salida, transferencia, ajuste).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
product_id	uuid	-	[x]	FK -> products(id)
lot_id	uuid	-	-	FK -> product_lots(id)
warehouse_id	uuid	-	[x]	FK -> warehouses(id)

Columna	Tipo	DEFAULT	NOT NULL	Comentario
type	text	-	[x]	CHECK: 'entry_purchase'
quantity	int	-	[x]	Cantidad (positiva o negativa según tipo)
reason	text	-	-	Motivo/razón del movimiento
reference	text	-	-	Referencia (ej. número factura, proforma)
user_id	uuid	-	-	FK -> users(id)
user_name	text	-	-	Nombre usuario (snapshot)
created_at	timestampz	now()	[x]	Timestamp

Enums:

- type: entry_purchase, exit_sale, transfer_out, transfer_in, adjustment_positive, adjustment_negative, return_in, return_out, quarantine, release, expiry, count_adjustment

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id)
- FOREIGN KEY: lot_id -> product_lots(id)
- FOREIGN KEY: warehouse_id -> warehouses(id)
- FOREIGN KEY: user_id -> users(id)
- CHECK: type in (...)
- RLS: inventory_movements_all (business_id=biz)

Índices:

- inventory_movements_business_idx: (business_id, created_at DESC)

Vista: inventory_stock_by_lot

Propósito: Vista desnormalizada de stock por lote.

```
SELECT
  business_id,
  branch_id,
  warehouse_id,
  product_id,
  id as lot_id,
  lot_number,
  expires_at,
  current_quantity as quantity,
  status
FROM product_lots
```

Propiedades: security_invoker = true (0008) para respetar RLS.

inventory_counts

Propósito: Conteos físicos de inventario.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
warehouse_id	uuid	-	[x]	FK -> warehouses(id) ON DELETE CASCADE
count_number	text	-	[x]	Número único conteo
count_type	text	-	[x]	CHECK: 'full'
status	text	'draft'	[x]	CHECK: 'draft'
assigned_to	uuid[]	'{}'	[x]	Array de usuarios asignados
started_at	timestampz	-	-	Inicio conteo
submitted_at	timestampz	-	-	Envío para revisión
reviewed_at	timestampz	-	-	Revisión completada
approved_at	timestampz	-	-	Aprobación completada
cancelled_at	timestampz	-	-	Cancelación
reviewed_by	uuid	-	-	FK -> users(id) (quien revisó)
approved_by	uuid	-	-	FK -> users(id) (quien aprobó)
notes	text	-	-	Notas
scan_count	int	0	[x]	Cantidad escaneos
item_count	int	0	[x]	Cantidad items
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Enums:

- count_type: full, partial, spot
- status: draft, in_progress, paused, submitted, reviewed, approved, rejected, adjusted, cancelled

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: warehouse_id -> warehouses(id) ON DELETE CASCADE
- FOREIGN KEY: reviewed_by -> users(id)
- FOREIGN KEY: approved_by -> users(id)
- UNIQUE: (business_id, count_number)
- CHECK: count_type in ('full','partial','spot')
- CHECK: status in (...)

- RLS: inventory_counts_all (business_id=biz)

inventory_count_scans

Propósito: Escaneos individuales durante conteo (offline-sync aware).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
inventory_count_id	uuid	-	[x]	FK -> inventory_counts(id) ON DELETE CASCADE
product_id	uuid	-	[x]	FK -> products(id)
product_lot_id	uuid	-	-	FK -> product_lots(id)
branch_id	uuid	-	[x]	FK -> branches(id)
warehouse_id	uuid	-	[x]	FK -> warehouses(id)
warehouse_location_id	uuid	-	-	Ubicación física
barcode	text	-	-	Código escaneado
scanned_quantity	int	1	[x]	Cantidad escaneada
scan_source	text	-	[x]	CHECK: 'camera\'
scanned_by	uuid	-	-	FK -> users(id)
scanned_by_name	text	-	-	Nombre usuario (snapshot)
scanned_at	timestamp tz	now()	[x]	Timestamp escaneo
device_id	text	-	[x]	ID dispositivo (para idempotencia offline)
offline_scan_id	text	-	[x]	ID escaneo offline (para idempotencia)
sync_status	text	'synced'	[x]	CHECK: 'synced\'
notes	text	-	-	Notas
created_at	timestamp tz	now()	[x]	Timestamp

Enums:

- scan_source: camera, bluetooth_scanner, manual
- sync_status: synced, pending, failed

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: inventory_count_id -> inventory_counts(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id)
- FOREIGN KEY: product_lot_id -> product_lots(id)
- FOREIGN KEY: branch_id -> branches(id)
- FOREIGN KEY: warehouse_id -> warehouses(id)

- FOREIGN KEY: scanned_by -> users(id)
- UNIQUE: (device_id, offline_scan_id) [idempotencia offline]
- CHECK: scan_source in ('camera','bluetooth_scanner','manual')
- CHECK: sync_status in ('synced','pending','failed')
- RLS: inventory_count_scans_all (business_id=biz)

Índices:

- inventory_count_scans_idempotent: (device_id, offline_scan_id) UNIQUE
- inventory_count_scans_count_idx: (inventory_count_id, scanned_at DESC)

inventory_count_items

Propósito: Resumen por producto/lote en un conteo (esperado vs. contado).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
inventory_count_id	uuid	-	[x]	FK -> inventory_counts(id) ON DELETE CASCADE
product_id	uuid	-	[x]	FK -> products(id)
product_sku	text	-	[x]	SKU (snapshot)
product_name	text	-	[x]	Nombre (snapshot)
product_lot_id	uuid	-	-	FK -> product_lots(id)
lot_number	text	-	-	Número lote (snapshot)
expires_at	date	-	-	Vencimiento (snapshot)
warehouse_id	uuid	-	[x]	FK -> warehouses(id)
expected_quantity	int	-	[x]	Cantidad esperada (sistema)
counted_quantity	int	0	[x]	Cantidad contada
difference_quantity	int	GENERATED	[x]	counted - expected (GENERATED ALWAYS AS ... STORED)
status	text	'match'	[x]	CHECK: 'match'
last_scan_at	timestamp tz	-	-	Último escaneo
created_at	timestamp tz	now()	[x]	Timestamp
updated_at	timestamp tz	now()	[x]	Timestamp

Enums:

- status: match, shortage, overage, expired, unregistered

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE

- FOREIGN KEY: inventory_count_id -> inventory_counts(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id)
- FOREIGN KEY: product_lot_id -> product_lots(id)
- FOREIGN KEY: warehouse_id -> warehouses(id)
- CHECK: status in ('match','shortage','overage','expired','unregistered')
- RLS: inventory_count_items_all (business_id=biz)

inventory_count_evidence

Propósito: Fotos/documentos de evidencia en conteos.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
inventory_count_id	uuid	-	[x]	FK -> inventory_counts(id) ON DELETE CASCADE
inventory_count_item_id	uuid	-	-	FK -> inventory_count_items(id)
file_url	text	-	[x]	URL archivo (Storage)
file_type	text	-	[x]	Tipo MIME (image/jpeg, etc.)
notes	text	-	-	Notas
uploaded_by	uuid	-	-	FK -> users(id)
created_at	timestamp tz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: inventory_count_id -> inventory_counts(id) ON DELETE CASCADE
- FOREIGN KEY: inventory_count_item_id -> inventory_count_items(id)
- FOREIGN KEY: uploaded_by -> users(id)
- RLS: inventory_count_evidence_all (business_id=biz)

inventory_count_sync_logs

Propósito: Trazabilidad de sincronización offline-online de conteos.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
inventory_count_id	uuid	-	[x]	FK -> inventory_counts(id)
device_id	text	-	[x]	ID dispositivo origen
user_id	uuid	-	-	FK -> users(id)
sync_status	text	-	[x]	CHECK: 'success'

Columna	Tipo	DEFAULT	NOT NULL	Comentario
request_payload	jsonb	-	-	Payload enviado
response_payload	jsonb	-	-	Respuesta servidor
conflict_detected	boolean	false	[x]	Conflicto detectado
error_message	text	-	-	Mensaje error
synced_at	timestampz	now()	[x]	Timestamp sincronización
created_at	timestampz	now()	[x]	Timestamp

Enums:

- sync_status: success, partial, failed

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: inventory_count_id -> inventory_counts(id)
- FOREIGN KEY: user_id -> users(id)
- CHECK: sync_status in ('success','partial','failed')
- RLS: inventory_count_sync_logs_all (business_id=biz)

lot_quarantine

Propósito: Cuarentena de lotes (problema detectado).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
product_lot_id	uuid	-	[x]	FK -> product_lots(id) ON DELETE CASCADE
reason	text	-	[x]	Motivo cuarentena
user_id	uuid	-	-	FK -> users(id) (quien quarantine)
released_at	timestampz	-	-	Timestamp de liberación
released_by	uuid	-	-	FK -> users(id) (quien libera)
created_at	timestampz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: product_lot_id -> product_lots(id) ON DELETE CASCADE
- FOREIGN KEY: user_id -> users(id)
- FOREIGN KEY: released_by -> users(id)
- RLS: lot_quarantine_all (business_id=biz)

lot_recalls

Propósito: Recalls de lotes (retirada de mercado).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
product_lot_id	uuid	-	[x]	FK -> product_lots(id) ON DELETE CASCADE
reason	text	-	[x]	Motivo recall
initiated_by	uuid	-	-	FK -> users(id)
customers_notified	int	0	[x]	Cantidad clientes notificados
created_at	timestamp z	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: product_lot_id -> product_lots(id) ON DELETE CASCADE
- FOREIGN KEY: initiated_by -> users(id)
- RLS: lot_recalls_all (business_id=biz)

D. CLIENTES (CRM)

clients

Propósito: Clientes/pacientes del negocio.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
customer_number	text	-	[x]	Número cliente único por business
first_name	text	-	[x]	Nombre
last_name	text	-	[x]	Apellido
document_type	text	-	-	CHECK: 'cedula\'
document_number	text	-	-	Número documento
phone	text	-	-	Teléfono
whatsapp	text	-	-	WhatsApp
email	text	-	-	Email
birth_date	date	-	-	Fecha nacimiento
address	text	-	-	Dirección
city	text	-	-	Ciudad
province	text	-	-	Provincia
source	text	'manual'	[x]	CHECK: 'manual\'

Columna	Tipo	DEFAULT	NOT NULL	Comentario
tags	text[]	'{}'	[x]	Etiquetas/categorías
default_billing_type	text	'consumo'	[x]	CHECK: 'consumo\'
skin_type	text	'not_specified'	[x]	CHECK: 'not_specified\'
total_spent	numeric(14,2)	0	[x]	Total gastado (desnormalizado)
total_orders	int	0	[x]	Cantidad órdenes (desnormalizado)
last_visit_at	timestamptz	-	-	Última visita
notes	text	-	-	Notas
consents	jsonb	'[]'	[x]	Consentimientos (marketing, etc.)
created_at	timestamptz	now()	[x]	Timestamp
updated_at	timestamptz	now()	[x]	Timestamp
deleted_at	timestamptz	-	-	Soft delete

Enums:

- document_type: cedula, rnc, passport
- source: manual, whatsapp, web, import, agendapro
- default_billing_type: consumo, credito_fiscal
- skin_type: not_specified, normal, dry, oily, combination, sensitive, acne_prone, mature, hyperpigmentation, rosacea_reactive

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, customer_number)
- UNIQUE (parcial): (business_id, document_number) WHERE document_number IS NOT NULL AND deleted_at IS NULL
- CHECK: document_type in ('cedula','rnc','passport')
- CHECK: source in ('manual','whatsapp','web','import','agendapro')
- CHECK: default_billing_type in ('consumo','credito_fiscal')
- CHECK: skin_type in ('not_specified','normal','dry','oily','combination','sensitive','acne_prone','mature','hyperpigmentation','rosacea_reactive')
- RLS: clients_sel/ins/upd/del (business_id=biz)

Índices:

- clients_business_document_unique: (business_id, document_number) WHERE document_number IS NOT NULL AND deleted_at IS NULL
- clients_business_idx: (business_id)
- clients_business_active_idx: (business_id, deleted_at)

E. DGII / FISCAL / POS

dgii_settings

Propósito: Configuración DGII por business (única por business).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE, UNIQUE
rnc_emisor	text	-	[x]	RNC emisor (espejo de businesses.rnc)
razon_social_emisor	text	-	[x]	Razón social (espejo de businesses.legal_name)
nombre_comercial	text	-	-	Nombre comercial (espejo de businesses.commercial_name)
direccion_emisor	text	-	[x]	Dirección fiscal
municipio	text	-	-	Municipio
provincia	text	-	-	Provincia
actividad_economica	text	-	-	Actividad económica DGII
telefono_emisor	text	-	-	Teléfono
correo_emisor	text	-	-	Email
website	text	-	-	Sitio web
ambiente	text	'testecf'	[x]	CHECK: 'testecf\
dgii_enabled_real_send	boolean	false	[x]	Habilita envío a ecf (no testecf). Gating crítico.
base_url_testecf	text	'https://ecf.dgii.gov.do/testecf'	[x]	URL testecf
base_url_certecf	text	'https://ecf.dgii.gov.do/certecf'	[x]	URL certecf
base_url_ecf	text	'https://ecf.dgii.gov.do/ecf'	[x]	URL ecf (producción)
default_cash_closing_ecf_percentage	numeric(5,2)	0	[x]	% default proformas -> e-CF en cierre (0-100)
allow_user_change_closing_percentage	boolean	false	[x]	Permite al usuario cambiar %
minimum_closing_ecf_percentage	numeric(5,2)	0	[x]	% mínimo permitido (0-100)
maximum_closing_ecf_percentage	numeric(5,2)	100	[x]	% máximo permitido (0-100)
require_admin_authorization_below_100_percent	boolean	true	[x]	Requiere autorización si % < 100
auto_generate_ecf_on_cash_closing	boolean	false	[x]	Auto-generar e-CF en cierre
applies_to_payment_methods	text[]	array['cash','transfer']	[x]	Métodos pago a los que aplica %
created_at	timestampz	now()	[x]	Timestamp

Columna	Tipo	DEFAULT	NOT NULL	Comentario
updated_at	timestamp	now()	[x]	Timestamp

Enums:

- ambiente: testecf, certecf, ecf

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: business_id
- CHECK: ambiente in ('testecf','certecf','ecf')
- CHECK: default_cash_closing_ecf_percentage BETWEEN 0 AND 100
- CHECK: minimum_closing_ecf_percentage BETWEEN 0 AND 100
- CHECK: maximum_closing_ecf_percentage BETWEEN 0 AND 100
- CHECK: minimum_closing_ecf_percentage <= maximum_closing_ecf_percentage (constraint dgii_settings_percentage_range)
- RLS: dgii_settings_all (business_id=biz)

dgii_certificates

Propósito: Certificados digitales X.509 para firma DGII (metadata + referencias seguras).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
alias	text	-	[x]	Alias legible (ej. 'Cert Principal 2024')
subject_dn	text	-	-	DN del subject
issuer_dn	text	-	-	DN del issuer
serial_number	text	-	-	Serial number
valid_from	timestamp	-	-	Fecha inicio validez
valid_to	timestamp	-	-	Fecha fin validez
pkcs12_storage_bucket	text	'certificates'	-	Bucket Storage (preferido)
pkcs12_storage_path	text	-	-	Ruta en Storage (si se usa bucket)
pkcs12_encrypted_blob	bytea	-	-	Blob cifrado AES-256-GCM (fallback)
kdf	text	-	-	Derivación de clave usada
iv	bytea	-	-	IV del cifrado

Columna	Tipo	DEFAULT	NOT NULL	Comentario
tag	bytea	-	-	Tag de autenticación
password_secret_ref	text	-	[x]	Referencia simbólica a secret (Vercel Env, Vault, KMS). NUNCA el valor en claro.
is_active	boolean	false	[x]	Certificado activo
revoked_at	timestampz	-	-	Timestamp revocación
uploaded_by	uuid	-	-	FK -> users(id)
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: uploaded_by -> users(id)
- EXCLUDE: (business_id WITH =) WHERE is_active = true [solo un cert activo por business]
- RLS: dgii_certificates_all (business_id=biz)

Índices:

- dgii_certificates_business_idx: (business_id)

ecf_sequences

Propósito: Secuencias e-NCF (rangos autorizados por DGII).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
tipo_ecf	text	-	[x]	CHECK: '31'
prefix	text	-	[x]	Prefijo (ej. 'E31')
range_start	int	-	[x]	Inicio rango
range_end	int	-	[x]	Fin rango
next_number	int	-	[x]	Próximo número a asignar
fecha_vencimiento	date	-	[x]	Vencimiento autorización DGII
ambiente	text	-	[x]	CHECK: 'testecf'
status	text	'active'	[x]	CHECK: 'active'
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Enums:

- tipo_ecf: 31, 32, 33, 34, 41, 43, 44, 45, 46, 47
 - 31: Factura de Crédito Fiscal
 - 32: Factura de Consumo
 - 33: Nota de Débito
 - 34: Nota de Crédito
 - 41-47: Otros tipos (reservados)
- ambiente: testecf, certecf, ecf
- status: active, expiring, exhausted, expired, cancelled

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, tipo_ecf, ambiente, range_start)
- CHECK: tipo_ecf in ('31','32','33','34','41','43','44','45','46','47')
- CHECK: ambiente in ('testecf','certecf','ecf')
- CHECK: status in ('active','expiring','exhausted','expired','cancelled')
- CHECK: range_end >= range_start (constraint ecf_sequences_range_ok)
- CHECK: next_number >= range_start AND next_number <= range_end + 1 (constraint ecf_sequences_next_ok)
- RLS: ecf_sequences_all (business_id=biz)

Índices:

- ecf_sequences_unique: (business_id, tipo_ecf, ambiente, range_start) UNIQUE
- ecf_sequences_business_idx: (business_id, tipo_ecf, ambiente, status)

Funciones:

- reserve_ecf_sequence_number(p_business_id uuid, p_tipo_ecf text, p_ambiente text) -> int: Reserva atómicamente el próximo número disponible. Incrementa next_number, marca como 'exhausted' si se agota. Devuelve NULL si no hay secuencia disponible. Usa FOR UPDATE para concurrencia.

payment_methods

Propósito: Formas de pago disponibles.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
code	text	-	[x]	Código único por business (ej. 'cash', 'card', 'transfer')
label	text	-	[x]	Etiqueta legible
requires_immediate_ecf	boolean	false	[x]	Si true -> e-CF inmediato, no proforma
default_ecf_type	text	-	-	CHECK: '31\'
is_active	boolean	true	[x]	Activo

Columna	Tipo	DEFAULT	NOT NULL	Comentario
created_at	timestamp_tz	now()	[x]	Timestamp
updated_at	timestamp_tz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, code)
- CHECK: default_ecf_type in ('31','32','33','34','41','43','44','45','46','47') (si no NULL)
- RLS: payment_methods_all (business_id=biz)

cash_registers

Propósito: Cajas/terminales POS.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
code	text	-	[x]	Código único por business (ej. 'CAJA-01')
name	text	-	[x]	Nombre (ej. 'Caja Principal')
is_active	boolean	true	[x]	Activa
created_at	timestamp_tz	now()	[x]	Timestamp
updated_at	timestamp_tz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- UNIQUE: (business_id, code)
- RLS: cash_registers_all (business_id=biz)

cash_register_sessions

Propósito: Sesiones de caja (abierta -> cerrada).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
cash_register_id	uuid	-	[x]	FK -> cash_registers(id) ON DELETE CASCADE

Columna	Tipo	DEFAULT	NOT NULL	Comentario
session_number	text	-	[x]	Número único por business
opened_by	uuid	-	[x]	FK -> users(id) (quien abrió)
opened_by_name	text	-	[x]	Nombre usuario (snapshot)
opened_at	timestamptz	now()	[x]	Timestamp apertura
opening_amount	numeric(14,2)	0	[x]	Monto inicial en caja
closed_by	uuid	-	-	FK -> users(id) (quien cerró)
closed_at	timestamptz	-	-	Timestamp cierre
expected_cash	numeric(14,2)	0	[x]	Efectivo esperado (cálculo sistema)
counted_cash	numeric(14,2)	-	-	Efectivo contado (manual)
difference_amount	numeric(14,2)	-	-	Diferencia (counted - expected)
totals	jsonb	'{}'	[x]	Totales por método pago (desnormalizado)
notes	text	-	-	Notas
status	text	'open'	[x]	CHECK: 'open\'
created_at	timestamptz	now()	[x]	Timestamp
updated_at	timestamptz	now()	[x]	Timestamp

Enums:

- status: open, closing, closed, reverted

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: cash_register_id -> cash_registers(id) ON DELETE CASCADE
- FOREIGN KEY: opened_by -> users(id)
- FOREIGN KEY: closed_by -> users(id)
- UNIQUE: (business_id, session_number)
- CHECK: status in ('open','closing','closed','reverted')
- RLS: cash_register_sessions_all (business_id=biz)

Índices:

- cash_register_sessions_business_idx: (business_id, status, opened_at DESC)

proformas

Propósito: Documentos de venta (proformas o facturas inmediatas, fiscales o no).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK

Columna	Tipo	DEFAULT	NOT NULL	Comentario
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
cash_register_session_id	uuid	-	-	FK -> cash_register_sessions(id) ON DELETE SET NULL
number	text	-	[x]	Número único por business
customer_id	uuid	-	-	FK -> clients(id)
customer_name	text	-	[x]	Nombre cliente (snapshot)
customer_phone	text	-	-	Teléfono cliente (snapshot)
customer_document	text	-	-	Documento cliente (snapshot)
cashier_id	uuid	-	[x]	FK -> users(id) (cajero)
cashier_name	text	-	[x]	Nombre cajero (snapshot)
billing_type	text	-	-	CHECK: 'consumo\'
document_kind	text	'proforma'	[x]	CHECK: 'proforma\'
ecf_type	text	-	-	CHECK: '31\'
sequence_type	text	-	-	CHECK: 'consumo\'
subtotal	numeric(14,2)	0	[x]	Subtotal
discount	numeric(14,2)	0	[x]	Descuento total
discount_percent	numeric(5,2)	-	-	% descuento
discount_amount	numeric(14,2)	-	-	\$ descuento
itbis	numeric(14,2)	0	[x]	ITBIS total
total	numeric(14,2)	0	[x]	Total
paid	numeric(14,2)	0	[x]	Pagado
balance	numeric(14,2)	0	[x]	Saldo pendiente
amount_received	numeric(14,2)	-	-	Monto recibido en POS
change_amount	numeric(14,2)	-	-	Cambio entregado
status	text	'draft'	[x]	CHECK: 'draft\'
notes	text	-	-	Notas
ecf_number	text	-	-	Número e-NCF (si se convierte)
electronic_invoice_id	uuid	-	-	FK -> electronic_invoices(id) (deferrable)
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Enums:

- billing_type: consumo, credito_fiscal
- document_kind: proforma, invoice
- ecf_type: 31, 32, 33, 34
- sequence_type: consumo, credito_fiscal
- status: draft, issued, paid, partially_paid, pending_ecf, pending_cash_closing, selected_for_ecf, ecf_generation_pending, converted_to_ecf, closed_without_ecf, cancelled, expired, voided

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: cash_register_session_id -> cash_register_sessions(id) ON DELETE SET NULL
- FOREIGN KEY: customer_id -> clients(id)
- FOREIGN KEY: cashier_id -> users(id)
- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE SET NULL (DEFERRABLE INITIALLY DEFERRED)
- UNIQUE: (business_id, number)
- CHECK: billing_type in ('consumo','credito_fiscal') (si no NULL)
- CHECK: document_kind in ('proforma','invoice')
- CHECK: ecf_type in ('31','32','33','34') (si no NULL)
- CHECK: sequence_type in ('consumo','credito_fiscal') (si no NULL)
- CHECK: status in (...)
- RLS: proformas_all (business_id=biz)

Índices:

- proformas_business_status_idx: (business_id, status, created_at DESC)
- proformas_session_idx: (business_id, cash_register_session_id)

proforma_items

Propósito: Líneas de una proforma.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
proforma_id	uuid	-	[x]	FK -> proformas(id) ON DELETE CASCADE
line_no	int	-	[x]	Número línea (orden)
product_id	uuid	-	-	FK -> products(id)
product_sku	text	-	[x]	SKU (snapshot)
product_name	text	-	[x]	Nombre (snapshot)

Columna	Tipo	DEFAULT	NOT NULL	Comentario
product_lot_id	uuid	-	-	FK -> product_lots(id)
lot_number	text	-	-	Número lote (snapshot)
kind	text	'bien'	[x]	CHECK: 'bien\'
quantity	numeric(14,2)	-	[x]	Cantidad
unit_price	numeric(14,4)	-	[x]	Precio unitario
itbis_rate	numeric(5,2)	0	[x]	% ITBIS
discount	numeric(14,2)	0	[x]	Descuento línea
subtotal	numeric(14,2)	-	[x]	Subtotal (qty * unit_price - discount)
itbis	numeric(14,2)	0	[x]	ITBIS línea
total	numeric(14,2)	-	[x]	Total línea
indicador_facturacion	text	-	-	Indicador facturación DGII (codificación pendiente)
created_at	timestampz	now()	[x]	Timestamp

Enums:

- kind: bien, servicio

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: proforma_id -> proformas(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id)
- FOREIGN KEY: product_lot_id -> product_lots(id)
- UNIQUE: (proforma_id, line_no)
- CHECK: kind in ('bien','servicio')
- RLS: proforma_items_all (business_id=biz)

proforma_payments

Propósito: Pagos registrados en una proforma.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
proforma_id	uuid	-	[x]	FK -> proformas(id) ON DELETE CASCADE
payment_method_id	uuid	-	-	FK -> payment_methods(id)
method_code	text	-	[x]	CHECK: 'cash\'

Columna	Tipo	DEFAULT	NOT NULL	Comentario
amount	numeric(14, 2)	-	[x]	Monto pagado (> 0)
reference	text	-	-	Referencia (ej. número autorizacion, recibo)
user_id	uuid	-	[x]	FK -> users(id)
user_name	text	-	[x]	Nombre usuario (snapshot)
created_at	timestamp tz	now()	[x]	Timestamp

Enums:

- method_code: cash, card, transfer, azul, cardnet, visanet, paypal, manual, other

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: proforma_id -> proformas(id) ON DELETE CASCADE
- FOREIGN KEY: payment_method_id -> payment_methods(id)
- FOREIGN KEY: user_id -> users(id)
- CHECK: method_code in ('cash','card','transfer','azul','cardnet','visanet','paypal','manual','other')
- CHECK: amount > 0
- RLS: proforma_payments_all (business_id=biz)

Índices:

- proforma_payments_proforma_idx: (business_id, proforma_id)

cash_closings

Propósito: Cierre de caja (registro de totales y decisión de % a e-CF).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	[x]	FK -> branches(id) ON DELETE CASCADE
cash_register_session_id	uuid	-	[x]	FK -> cash_register_sessions(id) ON DELETE CASCADE
closing_by	uuid	-	[x]	FK -> users(id) (quien cierra)
closing_by_name	text	-	[x]	Nombre usuario (snapshot)
closing_at	timestamp tz	now()	[x]	Timestamp cierre
total_cash	numeric(14, 2)	0	[x]	Total efectivo
total_transfer	numeric(14, 2)	0	[x]	Total transferencia
total_card	numeric(14, 2)	0	[x]	Total tarjeta

Columna	Tipo	DEFAULT	NOT NULL	Comentario
total_other	numeric(14,2)	0	[x]	Total otros métodos
total_general	numeric(14,2)	0	[x]	Total general
total_proformas_pending	numeric(14,2)	0	[x]	\$ proformas pendientes (cash/transfer)
count_proformas_pending	int	0	[x]	Cantidad proformas pendientes
applied_percentage	numeric(5,2)	0	[x]	% aplicado (0-100)
target_amount_to_ecf	numeric(14,2)	0	[x]	\$ objetivo a e-CF (calculated)
actual_amount_to_ecf	numeric(14,2)	0	[x]	\$ actual convertido a e-CF
count_proformas_convert	int	0	[x]	Cantidad proformas convertidas
count_proformas_left_pending	int	0	[x]	Cantidad proformas no convertidas
authorizer_user_id	uuid	-	-	FK -> users(id) (quien autoriza si % < 100)
authorizer_name	text	-	-	Nombre autorizador (snapshot)
comment	text	-	-	Comentario
status	text	'pending'	[x]	CHECK: 'pending'\
created_at	timestamp tz	now()	[x]	Timestamp
updated_at	timestamp tz	now()	[x]	Timestamp

Enums:

- status: pending, confirmed, reverted

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE CASCADE
- FOREIGN KEY: cash_register_session_id -> cash_register_sessions(id) ON DELETE CASCADE
- FOREIGN KEY: closing_by -> users(id)
- FOREIGN KEY: authorizer_user_id -> users(id)
- CHECK: applied_percentage BETWEEN 0 AND 100
- CHECK: status in ('pending','confirmed','reverted')
- RLS: cash_closings_all (business_id=biz)

Índices:

- cash_closings_business_idx: (business_id, closing_at DESC)
- cash_closings_session_idx: (business_id, cash_register_session_id)

cash_closing_sales

Propósito: Proformas seleccionadas en un cierre para conversión a e-CF.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
cash_closing_id	uuid	-	[x]	FK -> cash_closings(id) ON DELETE CASCADE
proforma_id	uuid	-	[x]	FK -> proformas(id) ON DELETE CASCADE
selected_for_ecf	boolean	false	[x]	Seleccionada para e-CF
converted_to_ecf_at	timestamp tz	-	-	Timestamp conversión
electronic_invoice_id	uuid	-	-	FK -> electronic_invoices(id) (deferrable)
selection_method	text	'manual'	[x]	CHECK: 'manual'
created_at	timestamp tz	now()	[x]	Timestamp

Enums:

- selection_method: manual, fifo, largest, smallest

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: cash_closing_id -> cash_closings(id) ON DELETE CASCADE
- FOREIGN KEY: proforma_id -> proformas(id) ON DELETE CASCADE
- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE SET NULL (DEFERRABLE INITIALLY DEFERRED)
- UNIQUE: (cash_closing_id, proforma_id)
- CHECK: selection_method in ('manual','fifo','largest','smallest')
- RLS: cash_closing_sales_all (business_id=biz)

Índices:

- cash_closing_sales_closing_idx: (business_id, cash_closing_id)

cash_closing_percentage_logs

Propósito: Auditoría inmutable del porcentaje ingresado en cierres sensibles.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
cash_closing_id	uuid	-	[x]	FK -> cash_closings(id) ON DELETE CASCADE
entered_by	uuid	-	[x]	FK -> users(id)
entered_by_name	text	-	[x]	Nombre usuario
entered_at	timestamp tz	now()	[x]	Timestamp entrada
percentage_entered	numeric(5,2)	-	[x]	% ingresado (0-100)

Columna	Tipo	DEFAULT	NOT NULL	Comentario
total_proformas_disponibles	numeric(14, 2)	-	[x]	\$ proformas disponibles
target_amount_calculado	numeric(14, 2)	-	[x]	\$ objetivo calculado
actual_amount_convertido	numeric(14, 2)	-	[x]	\$ actual convertido
count_convertidas	int	-	[x]	Cantidad convertidas
count_left_pending	int	-	[x]	Cantidad no convertidas
comment	text	-	-	Comentario
authorizer_user_id	uuid	-	-	FK -> users(id) (si requirió autorización)
authorizer_name	text	-	-	Nombre autorizador
final_status	text	-	[x]	CHECK: 'confirmed'

Enums:

- final_status: confirmed, rejected, reverted

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: cash_closing_id -> cash_closings(id) ON DELETE CASCADE
- FOREIGN KEY: entered_by -> users(id)
- FOREIGN KEY: authorizer_user_id -> users(id)
- CHECK: percentage_entered BETWEEN 0 AND 100
- CHECK: final_status in ('confirmed','rejected','reverted')
- RLS: cash_closing_percentage_logs_all (business_id=biz)

Índices:

- cash_closing_percentage_logs_closing_idx: (business_id, cash_closing_id, entered_at DESC)

proforma_to_ecf_logs

Propósito: Trazabilidad inmutable de conversión proforma -> e-CF.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
proforma_id	uuid	-	[x]	FK -> proformas(id) ON DELETE CASCADE
electronic_invoice_id	uuid	-	[x]	FK -> electronic_invoices(id) ON DELETE SET NULL (deferrable)
triggered_by	uuid	-	[x]	FK -> users(id)
triggered_by_name	text	-	[x]	Nombre usuario
cash_closing_id	uuid	-	-	FK -> cash_closings(id) ON DELETE SET NULL (contexto del cierre)
reason	text	-	-	Motivo conversión (ej. 'via cash closing', 'immediate')

Columna	Tipo	DEFAULT	NOT NULL	Comentario
created_at	timestampz	now()	[x]	Timestamp

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: proforma_id -> proformas(id) ON DELETE CASCADE
- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE SET NULL (DEFERRABLE INITIALLY DEFERRED)
- FOREIGN KEY: triggered_by -> users(id)
- FOREIGN KEY: cash_closing_id -> cash_closings(id) ON DELETE SET NULL
- RLS: proforma_to_ecf_logs_all (business_id=biz)

Índices:

- proforma_to_ecf_logs_proforma_idx: (business_id, proforma_id)

electronic_invoices

Propósito: Facturas electrónicas DGII (e-CF).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
branch_id	uuid	-	-	FK -> branches(id) ON DELETE SET NULL
proforma_id	uuid	-	-	FK -> proformas(id) ON DELETE SET NULL
source_invoice_id	uuid	-	-	FK a otra e-CF (para NC/ND 33/34)
tipo_ecf	text	-	[x]	CHECK: '31\'
e_ncf	text	-	[x]	Número e-NCF (único por business + ambiente)
secuencia_id	uuid	-	-	FK -> ecf_sequences(id)
status	text	'draft'	[x]	CHECK: 'draft\'
customer_id	uuid	-	-	FK -> clients(id)
customer_name	text	-	-	Nombre cliente
customer_rnc	text	-	-	RNC cliente
subtotal_gravado	numeric(14, 2)	0	[x]	Subtotal gravado (ITBIS)
subtotal_exento	numeric(14, 2)	0	[x]	Subtotal exento
total_itbis	numeric(14, 2)	0	[x]	Total ITBIS
total_otros_impuestos	numeric(14, 2)	0	[x]	Otros impuestos
total	numeric(14, 2)	-	[x]	Total
currency	text	'DOP'	[x]	Moneda
xml_generated_path	text	-	-	Ruta XML generado (Storage)

Columna	Tipo	DEFAULT	NOT NULL	Comentario
xml_signed_path	text	-	-	Ruta XML firmado (Storage)
xml_response_path	text	-	-	Ruta respuesta XML (Storage)
pdf_path	text	-	-	Ruta PDF (Storage)
qr_code_payload	text	-	-	Payload QR
security_code	text	-	-	Código de seguridad DGII
hash_sha256	text	-	-	Hash SHA-256 documento
track_id	text	-	-	TrackId DGII
dgii_status_code	text	-	-	Código estado DGII
dgii_status_message	text	-	-	Mensaje estado DGII
ambiente	text	-	[x]	CHECK: 'testecf'
generated_at	timestampz	-	-	Timestamp generación XML
signed_at	timestampz	-	-	Timestamp firma
sent_at	timestampz	-	-	Timestamp envío a DGII
accepted_at	timestampz	-	-	Timestamp aceptación DGII
rejected_at	timestampz	-	-	Timestamp rechazo DGII
cancelled_at	timestampz	-	-	Timestamp cancelación
generated_by	uuid	-	-	FK -> users(id)
sent_by	uuid	-	-	FK -> users(id)
created_at	timestampz	now()	[x]	Timestamp
updated_at	timestampz	now()	[x]	Timestamp

Enums:

- tipo_ecf: 31, 32, 33, 34, 41, 43, 44, 45, 46, 47
- status: draft, generated, validated, signed, submitted, in_process, accepted, accepted_conditional, rejected, cancelled, error, voided
- ambiente: testecf, certecf, ecf

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: branch_id -> branches(id) ON DELETE SET NULL
- FOREIGN KEY: proforma_id -> proformas(id) ON DELETE SET NULL
- FOREIGN KEY: customer_id -> clients(id)
- FOREIGN KEY: secuencia_id -> ecf_sequences(id)
- FOREIGN KEY: generated_by -> users(id)
- FOREIGN KEY: sent_by -> users(id)
- UNIQUE: (business_id, e_ncf, ambiente)

- CHECK: tipo_ecf in ('31','32','33','34','41','43','44','45','46','47')
- CHECK: status in (...)
- CHECK: ambiente in ('testecf','certecf','ecf')
- RLS: electronic_invoices_all (business_id=biz)

Índices:

- electronic_invoices_business_status_idx: (business_id, status, created_at DESC)
- electronic_invoices_business_track_idx: (business_id, track_id) WHERE track_id IS NOT NULL
- electronic_invoices_proforma_idx: (business_id, proforma_id)

electronic_invoice_items

Propósito: Líneas de una factura electrónica (snapshot inmutable).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[X]	PK
business_id	uuid	-	[X]	FK -> businesses(id) ON DELETE CASCADE
electronic_invoice_id	uuid	-	[X]	FK -> electronic_invoices(id) ON DELETE CASCADE
line_no	int	-	[X]	Número línea
product_id	uuid	-	-	FK -> products(id) ON DELETE SET NULL
product_sku	text	-	-	SKU (snapshot)
name_item	text	-	[X]	Nombre item
description_item	text	-	-	Descripción
kind	text	'bien'	[X]	CHECK: 'bien\'
quantity	numeric(14,2)	-	[X]	Cantidad
unit_measure	text	-	-	Unidad medida
unit_price	numeric(14,4)	-	[X]	Precio unitario
discount_amount	numeric(14,2)	0	[X]	Descuento línea
itbis_rate	numeric(5,2)	0	[X]	% ITBIS
indicador_facturacion	text	-	-	Indicador facturación DGII
monto_item	numeric(14,2)	-	[X]	Monto total línea
created_at	timestampz	now()	[X]	Timestamp

Enums:

- kind: bien, servicio

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE

- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE CASCADE
- FOREIGN KEY: product_id -> products(id) ON DELETE SET NULL
- UNIQUE: (electronic_invoice_id, line_no)
- CHECK: kind in ('bien','servicio')
- RLS: electronic_invoice_items_all (business_id=biz)

dgii_submissions

Propósito: Registro de intentos de envío a DGII (immutable).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
electronic_invoice_id	uuid	-	[x]	FK -> electronic_invoices(id) ON DELETE CASCADE
attempt_no	int	1	[x]	Número intento (1..N)
endpoint_url	text	-	[x]	URL endpoint DGII usado
request_headers	jsonb	-	-	Headers request
request_body_path	text	-	-	Ruta XML request (Storage)
response_status	int	-	-	HTTP status response
response_body_path	text	-	-	Ruta XML response (Storage)
track_id	text	-	-	TrackId devuelto
error_code	text	-	-	Código error (si aplica)
error_message	text	-	-	Mensaje error
sent_at	timestamp tz	now()	[x]	Timestamp envío
responded_at	timestamp tz	-	-	Timestamp respuesta

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE CASCADE
- UNIQUE: (electronic_invoice_id, attempt_no)
- RLS: dgii_submissions_all (business_id=biz)

Índices:

- dgii_submissions_invoice_idx: (business_id, electronic_invoice_id, sent_at DESC)

dgii_status_logs

Propósito: Consultas de estado a DGII (immutable).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
electronic_invoice_id	uuid	-	[x]	FK -> electronic_invoices(id) ON DELETE CASCADE
track_id	text	-	[x]	TrackId consultado
response_status	int	-	-	HTTP status
response_code	text	-	-	Código estado DGII
response_message	text	-	-	Mensaje estado
response_body_path	text	-	-	Ruta respuesta (Storage)
consulted_at	timestamp tz	now()	[x]	Timestamp consulta

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: electronic_invoice_id -> electronic_invoices(id) ON DELETE CASCADE
- RLS: dgii_status_logs_all (business_id=biz)

Índices:

- dgii_status_logs_invoice_idx: (business_id, electronic_invoice_id, consulted_at DESC)
- dgii_status_logs_track_idx: (business_id, track_id)

dgii_received_ecf

Propósito: Facturas electrónicas RECIBIDAS de otros emisores (aprobación comercial).

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[x]	PK
business_id	uuid	-	[x]	FK -> businesses(id) ON DELETE CASCADE
tipo_ecf	text	-	[x]	CHECK: '31\'
e_ncf	text	-	[x]	Número e-NCF
rnc_emisor_sender	text	-	[x]	RNC del emisor
razon_social_emisor	text	-	-	Razón social emisor
xml_path	text	-	[x]	Ruta XML (Storage)
total	numeric(14, 2)	0	[x]	Total
received_at	timestamp tz	now()	[x]	Timestamp recepción
processed_at	timestamp tz	-	-	Timestamp procesamiento
commercial_approval_statuses	text	'pending'	[x]	CHECK: 'pending\'

Enums:

- tipo_ecf: 31, 32, 33, 34, 41, 43, 44, 45, 46, 47
- commercial_approval_status: pending, approved, conditionally_approved, rejected, total_rejection

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- UNIQUE: (business_id, e_ncf, rnc_emisor_sender)
- CHECK: tipo_ecf in ('31','32','33','34','41','43','44','45','46','47')
- CHECK: commercial_approval_status in ('pending','approved','conditionally_approved','rejected','total_rejection')
- RLS: dgii_received_ecf_all (business_id=biz)

Índices:

- dgii_received_ecf_business_idx: (business_id, received_at DESC)

dgii_commercial_approvals

Propósito: Decisión de aprobación comercial de facturas recibidas.

Columna	Tipo	DEFAULT	NOT NULL	Comentario
id	uuid	gen_random_uuid()	[X]	PK
business_id	uuid	-	[X]	FK -> businesses(id) ON DELETE CASCADE
received_ecf_id	uuid	-	[X]	FK -> dgii_received_ecf(id) ON DELETE CASCADE
decision	text	-	[X]	CHECK: 'approved'
decided_by	uuid	-	[X]	FK -> users(id)
decided_by_name	text	-	[X]	Nombre usuario
decided_at	timestamp tz	now()	[X]	Timestamp decisión
reason	text	-	-	Motivo rechazo/condiciones
xml_path	text	-	-	Ruta XML respuesta (Storage)
response_sent_at	timestamp tz	-	-	Timestamp envío respuesta a emisor
response_track_id	text	-	-	TrackId respuesta DGII

Enums:

- decision: approved, conditionally_approved, rejected, total_rejection

Constraints:

- PRIMARY KEY: id
- FOREIGN KEY: business_id -> businesses(id) ON DELETE CASCADE
- FOREIGN KEY: received_ecf_id -> dgii_received_ecf(id) ON DELETE CASCADE
- FOREIGN KEY: decided_by -> users(id)
- CHECK: decision in ('approved','conditionally_approved','rejected','total_rejection')
- RLS: dgii_commercial_approvals_all (business_id=biz)

Índices:

- dgii_commercial_approvals_business_idx: (business_id, decided_at DESC)

RESUMEN DE FUNCIONES PL/pgSQL

Función	Parámetros	Retorna	Propósito	Archivo
auth_business_id()	-	uuid	Lee business_id del JWT (root -> app_metadata -> user_metadata)	0001, 0006
auth_is_platform_admin()	-	boolean	Lee is_platform_admin del JWT	0001, 0006
select_lot_for_sale()	p_business_id, p_product_id, p_branch_id	uuid	Selecciona lote FEFO disponible para venta	0002
reserve_ecf_sequence_number()	p_business_id, p_tipo_ecf, p_ambiente	int	Reserva atómicamente próximo número e-NCF, incrementa next_number	0003

ENUMERACIONES SINTETIZADAS

Todas las columnas TEXT que actúan como enum:

Tabla	Columna	Valores	Notas
businesses	status	active, suspended, trial, past_due	Estado subscripción
branches	status	active, inactive	Estado sucursal
users	role	admin, manager, cashier, inventory, supervisor, auditor	Rol de usuario
users	status	active, invited, disabled	Estado usuario
clients	document_type	cedula, rnc, passport	Tipo documento
clients	source	manual, whatsapp, web, import, agendapro	Origen cliente
clients	default_billing_type	consumo, credito_fiscal	Tipo facturación default
clients	skin_type	not_specified, normal, dry, oily, combination, sensitive, acne_prone, mature, hyperpigmentation, rosacea_reactive	Tipo piel (dermatología)
product_lots	status	available, quarantine, expired, recalled, damaged, returned	Estado lote
inventory_movements	type	entry_purchase, exit_sale, transfer_out, transfer_in, adjustment_positive, adjustment_negative, return_in, return_out, quarantine, release, expiry, count_adjustment	Tipo movimiento
inventory_counts	count_type	full, partial, spot	Tipo conteo
inventory_counts	status	draft, in_progress, paused, submitted, reviewed, approved, rejected, adjusted, cancelled	Estado conteo
inventory_count_scans	scan_source	camera, bluetooth_scanner, manual	Origen escaneo
inventory_count_scans	sync_status	synced, pending, failed	Estado sincronización
inventory_count_items	status	match, shortage, overage, expired, unregistered	Resultado conteo

Tabla	Columna	Valores	Notas
inventory_count_sync_logs	sync_status	success, partial, failed	Resultado sincronización
dgii_settings	ambiente	testecf, certecf, ecf	Ambiente DGII
ecf_sequences	tipo_ecf	31, 32, 33, 34, 41, 43, 44, 45, 46, 47	Tipo comprobante e-CF (codificación DGII)
ecf_sequences	ambiente	testecf, certecf, ecf	Ambiente
ecf_sequences	status	active, expiring, exhausted, expired, cancelled	Estado secuencia
payment_methods	default_ecf_type	31, 32, 33, 34, 41, 43, 44, 45, 46, 47	Tipo e-CF default
cash_register_sessions	status	open, closing, closed, reverted	Estado sesión caja
proformas	billing_type	consumo, credito_fiscal	Tipo facturación
proformas	document_kind	proforma, invoice	Tipo documento (proforma no fiscal vs invoice fiscal)
proformas	ecf_type	31, 32, 33, 34	Tipo e-CF (si aplica)
proformas	sequence_type	consumo, credito_fiscal	Tipo secuencia
proformas	status	draft, issued, paid, partially_paid, pending_ecf, pending_cash_closing, selected_for_ecf, ecf_generation_pending, converted_to_ecf, closed_without_ecf, cancelled, expired, voided	Estado proforma
proforma_items	kind	bien, servicio	Tipo línea
proforma_payments	method_code	cash, card, transfer, azul, cardnet, visanet, paypal, manual, other	Método pago
cash_closings	status	pending, confirmed, reverted	Estado cierre
cash_closing_percentage_logs	final_status	confirmed, rejected, reverted	Resultado porcentaje
electronic_invoices	tipo_ecf	31, 32, 33, 34, 41, 43, 44, 45, 46, 47	Tipo comprobante
electronic_invoices	status	draft, generated, validated, signed, submitted, in_process, accepted, accepted_conditional, rejected, cancelled, error, voided	Estado e-CF
electronic_invoices	ambiente	testecf, certecf, ecf	Ambiente
electronic_invoice_items	kind	bien, servicio	Tipo línea
dgii_received_ecf	tipo_ecf	31, 32, 33, 34, 41, 43, 44, 45, 46, 47	Tipo
dgii_received_ecf	commercial_approval_status	pending, approved, conditionally_approved, rejected, total_rejection	Estado aprobación
dgii_commercial_approvals	decision	approved, conditionally_approved, rejected, total_rejection	Decisión

NOTAS FINALES

- 1. RLS: Todas las tablas de negocio tienen RLS habilitado. Las políticas se consolidaron en 0008-0009 para evitar "multiple permissive policies" y mejorar plan InitPlan.
- 2. Foreign Keys Circulares (Deferrable):

- `proformas.electronic_invoice_id -> electronic_invoices(id)`
- `cash_closing_sales.electronic_invoice_id -> electronic_invoices(id)`
- `proforma_to_ecf_logs.electronic_invoice_id -> electronic_invoices(id)`

Todas declaradas DEFERRABLE INITIALLY DEFERRED para permitir inserciones bidireccionales en la misma transacción.

- 3. Soft Deletes: businesses, branches, users, products, clients tienen columna `deleted_at`. Índices parciales respetan `WHERE deleted_at IS NULL`.
- 4. Desnormalizaciones Intencionales:
 - `brands.product_count` (actualización manual o trigger futuro)
 - `clients.total_spent, total_orders` (agregados de auditoría)
 - `proformas/proforma_items`: snapshots de producto/cliente en momento de venta
- 5. Storage: Rutas de archivos en columnas `*_path` (XML, PDF, respuestas DGII) refieren a Supabase Storage privado o similar. Nunca se almacenan binarios en la DB.
- 6. Funciones Seguras: `auth_business_id()`, `auth_is_platform_admin()`, `select_lot_for_sale()`, `reserve_ecf_sequence_number()` tienen `search_path` fijo (0008) para evitar secuestro de esquema.
- 7. Permisos Granulares: Los 18 permisos DGII/cash (creados en 0004-0005) mapean a un sistema RBAC en tablas `roles` y `role_permissions`. Las políticas RLS siguen siendo a nivel de `tenant (business_id)`; los permisos granulares se evalúan en aplicación.
- 8. No Hay Datos Reales DGII: Certificados, secuencias, RNC emisor no se insertan en migraciones. El usuario debe importarlos manualmente después de autorizar Fase C.

Este esquema es exhaustivo, preciso y listo para reconstrucción desde cero.

4. Seguridad, RLS, roles y autenticacion

4.1 Funciones SQL de auth

Definidas en 0001_phase1_core.sql y actualizadas en 0006_auth_helpers_jwt_metadata.sql. Leen los claims del JWT con una cascada de prioridad: raiz -> app_metadata -> user_metadata. En 0008 ambas reciben set search_path = public, auth, extensions (anti path-injection).

```
create or replace function public.auth_business_id()
returns uuid language sql stable as $$
select coalesce(
    nullif(current_setting('request.jwt.claims', true)::jsonb -> 'business_id', ''),
    nullif(current_setting('request.jwt.claims', true)::jsonb -> 'app_metadata' -> 'business_
id', ''),
    nullif(current_setting('request.jwt.claims', true)::jsonb -> 'user_metadata' -> 'business
_id', '')
)::uuid;
$$;

create or replace function public.auth_is_platform_admin()
returns boolean language sql stable as $$
select coalesce(
    (current_setting('request.jwt.claims', true)::jsonb -> 'is_platform_admin')::boolean,
    (current_setting('request.jwt.claims', true)::jsonb -> 'app_metadata' -> 'is_platform_adm
in')::boolean,
    (current_setting('request.jwt.claims', true)::jsonb -> 'user_metadata' -> 'is_platform_ad
min')::boolean,
    false
);
$$;
```

4.2 Patron RLS

Cada tabla de negocio tiene RLS habilitada. El patron dominante para tablas con business_id es una politica FOR ALL a rol authenticated:

```
create policy <tabla>_all on <tabla>
for all to authenticated
using      (business_id = (select public.auth_business_id()))
with check (business_id = (select public.auth_business_id()));
```

Tablas core (businesses, branches, users, clients, plans, audit_logs) usan policias separadas por comando (SELECT/INSERT/UPDATE/DELETE) con variaciones para super admin. Catalogos globales (permissions, roles, role_permissions) tienen _read_all (SELECT: true). El envolver auth_() en (select ...) es la optimizacion "InitPlan" aplicada en 0008/0009 (evalua la funcion una vez por query, no por fila). 0007 agrega INSERT a audit_logs con guardia de user_id.

4.3 Roles (7) y permisos (18)

Tablas: roles(code PK, label, description), permissions(code PK, module, description, is_destructive), role_permissions(role_code, permission_code, PK compuesta).

Rol	Permisos	Resumen
super_admin	18 (todos)	dgii:* + cash:*

Rol	Permisos	Resumen
admin	18 (todos)	dgii:* + cash:*
manager	12	e-CF (6) + NC + reports:view + cash open/close/change_pct
cashier	4	generate_xml, download_pdf, cash open/close
inventory	0	sin permisos DGII/cash
supervisor	3	reports:view, cash authorize_below_100, reverse_closing
auditor	4	reports:view + invoices check_status/download_xml/download_pdf

Total: 59 asignaciones (role_code, permission_code). Los permisos se sincronizan con roleDefinitions en TypeScript via el test role-permissions-sync.test.ts. Lista de permisos: dgii:configure, dgii:certificate:upload, dgii:sequences:manage, dgii:invoices:{generate_xml,validate_xml,sign,send,check_status,download_xml,download_pdf}, dgii:credit_notes:create, dgii:reports:view, dgii:certification:run_tests, cash:{open,close,change_closing_percentage,authorize_below_100_percent,reverse_closing}.

4.4 JWT, sesion y clientes Supabase

Claims usados: business_id, is_platform_admin, y en user_metadata: branch_id, role, full_name, branch_ids. La sesion server-side se resuelve en src/server/auth/context.ts (getSession()), que en modo mock devuelve un usuario hardcoded y en modo supabase lee el JWT de las cookies via supabase.auth.getUser(). No usa AsyncLocalStorage: el contexto (RepoContext con businessId/branchId/userId) se pasa explicitamente a los repositorios.

```
// Tres clientes Supabase (src/lib/supabase/)
createServer()           // anon key + cookies -> RLS activo (SC, server actions, route handlers)
createServiceRoleClient() // service_role      -> BYPASSA RLS (solo backend, verificar business_id)
createClient()           // anon key (browser) -> RLS como limite de seguridad
```

Middleware (src/middleware.ts): refresca la sesion en cada request, redirige a /login si la ruta protegida no tiene sesion, y bloquea /super-admin/* si is_platform_admin !== true. Rutas publicas: /login, /auth/callback, /api/whatsapp/webhook, /api/health, assets.

Auth actions (src/server/auth/actions.ts): signIn(formData) (Zod email+password>=8; mock setea cookie dl-mock-session 8h; supabase signInWithPassword + deteccion MFA), verifyMfa(code) (TOTP), signOut().

5. Motor de comprobante fiscal (e-CF)

Servicios en apps/web/src/server/services/dgii/ (funciones puras server-only). Pipeline canonico: build XML -> sign -> validate XSD -> security code + QR -> PDF.

5.1 Estructura del XML por tipo (orden exacto del builder)

El builder (builder.ts) produce el XML en el orden que exige el XSD oficial. Esqueleto comun:

```
<ECF>
  <Encabezado>
    <Version>1.0</Version>
    <IdDoc> ... </IdDoc>
    <Emisor> ... </Emisor>
    <Comprador> ... </Comprador>
    <Totales> ... </Totales>
  </Encabezado>
  <DetallesItems><Item> ... </Item> x N</DetallesItems>
  <InformacionReferencia> ... </InformacionReferencia> <!-- obligatorio en 33/34 -->
  <FechaHoraFirma>dd-MM-yyyy HH:mm:ss</FechaHoraFirma>
  <Signature> ... </Signature> <!-- XMLDSig enveloped, tras firmar -->
>
</ECF>
```

Bifurcaciones por tipo (seccion IdDoc):

Tip o	Comprobante	Particularidad
31	Credito Fiscal	emite FechaVencimientoSecuencia; RNC+RazonSocial comprador OBLIGATORIOS
32	Consumo	OMITE FechaVencimientoSecuencia; comprador OPCIONAL (consumidor final)
33	Nota de Debito	emite FechaVencimientoSecuencia; InformacionReferencia OBLIGATORIA
34	Nota de Credito	reemplaza por IndicadorNotaCredito (0/1, OBLIGATORIO); InformacionReferencia OBLIGATORIA; TipoIngresos opcional

Totales (orden estricto, todos opcionales menos MontoTotal): MontoGravadoTotal, MontoGravadoI1/I2/I3, MontoExento, ITBIS1/2/3, TotalITBIS, TotalITBIS1/2/3, MontoTotal (obligatorio). I1=18%, I2=16%, I3=0%.

Item: NumeroLinea(1..1000), IndicadorFacturacion(0-4), NombreItem, IndicadorBienoServicio(1=bien, 2=servicio), DescripcionItem?, CantidadItem(>0), UnidadMedida?, PrecioUnitarioItem(.toFixed(4)), DescuentoMonto?, MontoItem.

InformacionReferencia (33/34): NCFModificado(11..19), RNCOtroContribuyente(9/11), FechaNCFModificado, CodigoModificacion(1=Anulacion,2=Cambios,3=Devolucion,4=Descuento,5=Correccion).

5.2 Tipo de entrada (TypeScript)

```
interface EcfBuilderInput {
  tipoEcf: "31"|"32"|"33"|"34"; // 41+ lanzan EcfBuilderUnsupported
  eNcf: string; // 13 chars alfanumericos
  fechaVencimientoSecuencia: Date; // 31/33; omitido 32; reemplazado en 34
  indicadorNotaCredito?: 0|1; // OBLIGATORIO si tipoEcf=="34"
  tipoIngresos: "01".."06";
  tipoPago: 1|2|3; // 1=contado, 2=credito, 3=gratis
```

```

    indicadorMontoGravado?: 0|1;      // default 0 (lineas SIN ITBIS)
    formasPago?: { formaPago:1..8; montoPago:number }[]; // 1..7 items
    emisor: { rncEmisor; razonSocialEmisor; nombreComercial?; sucursal?; direccionEmisor;
              municipio?; provincia?; telefonosEmisor?: string[]; correoEmisor?; website?;
              actividadEconomica?; fechaEmision: Date };
    comprador: { rncComprador?; razonSocialComprador?; contactoComprador?; correoComprador?;
                 direccionComprador?; municipioComprador?; provinciaComprador? };
    totales: { montoGravadoTotal?; montoGravadoI1?..I3?; montoExento?; itbis1?..3?;
               totalItbis?; totalItbis1?..3?; montoTotal }; // montoTotal obligatorio
    items: EcfItem[]; // 1..1000
    informacionReferencia?: { ncfModificado; rncOtroContribuyente; fechaNCFModificado;
                              codigoModificacion: 1..5 }; // obligatorio 33/34
    fechaHoraFirma: Date;
  }

```

5.3 Firma XMLDSig (signer.ts)

Libreria xml-crypto (SignedXml). Firma enveloped insertada como ultimo hijo de <ECF>.

Parametro	Valor
SignatureMethod	http://www.w3.org/2001/04/xmldsig-more#rsa-sha256
DigestMethod	http://www.w3.org/2001/04/xmldsig-core#sha256
Canonicalizacion	http://www.w3.org/TR/2001/REC-xml-c14n-20010315
Transform	enveloped-signature + C14N
Reference URI	"" (isEmptyUri: true) - documento entero
KeyInfo	X509Data -> X509Certificate (base64 sin cabeceras)
XPath ancla	//*[local-name(.)='ECF'] ; location append

La clave privada (privateKeyPem) nunca persiste ni se cachea; se descifra en memoria por el caller. Errores genericos (DgiiSignerError) sin material sensible. verifyEcfSignature() para roundtrip/tests.

5.4 Validacion XSD (validator.ts)

xmllint-wasm (libxml2 en WebAssembly) contra los XSD oficiales en docs/dgii/xsd/. Parches en memoria: (1) strip BOM UTF-8; (2) typo del XSD 31 name="IndicadorServicioTodoIncluidoType" (espacio inicial; 32/33/34 limpios). Regla critica: el XSD exige un xs:any final (la <Signature>), por lo que un XML SIN firmar FALLA la validacion. Orden correcto: build -> sign -> validate.

5.5Codigo de seguridad y QR

- security-code.ts: extrae <SignatureValue>, limpia a alfanumericos y toma los primeros 8.

(Heuristica; el algoritmo oficial DGII esta pendiente de validar - matriz D-06.)

- qr.ts: URL https://ecf.dgii.gov.do/{testecf|certecf}/ConsultaTimbre/api/Consulta con params

RncEmisor, RncComprador?, ENCF, FechaEmision(dd-MM-yyyy), MontoTotal(.toFixed 2),
CodigoSeguridadleCF. qrcode genera PNG/SVG/dataURL.

5.6 PDF (pdf.ts)

PDFKit, tamano LETTER, margen 50, fuentes Helvetica. Secciones: encabezado (emisor, RNC, tipo, e-NCF, fecha), comprador (o "Consumidor Final"), tabla de items, totales, footer (estado, ambiente, TrackId, codigo de seguridad) y QR 100x100. Devuelve Buffer (no escribe a disco).

5.7 Mapeos y reglas fiscales

- `proforma-to-input.ts`: convierte una proforma POS en `EcfBuilderInput`. Deriva

`indicadorFacturacion` de `itbisRate` (18->1, 16->2, 0->4), separa precio pre-ITBIS, sintetiza eNCF (demo). En produccion el eNCF viene de la reserva atomica de secuencias.

- `source-invoice-to-nc.ts`: genera la Nota de Credito (34) desde un e-CF origen.

`indicadorNotaCredito` = (dias > 30 ? 1 : 0); item unico con el motivo; `InformacionReferencia` apunta al e-NCF original con `codigoModificacion`.

Regla	Valor
ITBIS	18% / 16% / 0% (exento)
Indicador facturacion	0 NoFacturable, 1 ITBIS18, 2 ITBIS16, 3 ITBIS0, 4 Exento
Decimales	montos <code>.toFixed(2)</code> ; precio unitario <code>.toFixed(4)</code>
Fechas	dd-MM-yyyy ; firma dd-MM-yyyy HH:mm:ss
RNC	<code>/^(?:\d{9})</code>
eNCF	13 chars alfanumericos
Items	1..1000 ; cantidad > 0
Formas de pago	1..7
Telefonos	ddd-ddd-dddd (max 3)

5.8 Dry-run y estado del envio

`testecf-client.ts`: `prepareTestecfSubmission()` ejecuta `build->sign->validate->verify` LOCAL y devuelve evidencia (endpoints que se invocarian, validacion XSD, firma verificada, tamanos, base64) SIN hacer HTTP. `executeTestecfSubmission()` es un stub que SIEMPRE lanza `TestecfSendDisabled` con la lista de gates pendientes (DGII_TESTECF_SEND_ENABLED, postulacion, rango, confirmacion del usuario). La autenticacion DGII (semilla->firma->token), el cliente HTTP real y el polling de `TrackId` NO estan implementados (Fases G/H).

6. API y contratos

6.1 Routing (App Router)

Route groups: (app) (negocio, requiere auth), (super-admin) (plataforma), api/ (route handlers), login/.
Dentro de (app): admin/*, caja, clientes, conteo-fisico, devoluciones, dgii, inventario, notas-credito, pagos, pos, productos, proformas, reportes, ventas, whatsapp, ia.

6.2 Route handlers (endpoints)

Endpoint	Metodo	Auth	Proposito
/api/health	GET	no	Liveness + estado de integraciones
/api/dgii/certificacion/run-test	POST	no (demo)	Pipeline build->sign->QR->PDF con cert dummy
/api/dgii/certificate/current	GET	si	Certificado activo (sin material sensible)
/api/dgii/certificate/test-local	POST	si	Descifra .p12 y prueba firma/validacion local
/api/dgii/facturas/[id]/pdf	GET	no (demo)	PDF de e-CF mock
/api/dgii/facturas/[id]/xml-signed	GET	no (demo)	XML firmado mock
/api/dgii/facturas/[id]/xml-unsigned	GET	no (demo)	XML sin firmar mock
/api/dgii/invoices/testecf-send	POST	si	Dry-run Fase G (NO envia a DGII)
/api/dgii/notas-credito/create	POST	no (demo)	Crea NC (34) desde factura origen
/api/dgii/preview/pdf	xml-signed	xml-unsigned	POST
/api/inventory-counts/sync	POST	si	Sync idempotente de escaneos offline
/api/whatsapp/webhook	GET/POST	no	Handshake + eventos Meta

Patron de respuesta de error: 400 (json/parametros invalidos), 401 (unauthenticated), 403/503 (feature_disabled), 404 (no encontrado), 412 (precondicion: sin certificado/settings), 422 (test_failed), 500 (unknown_error). Endpoints de archivos devuelven Content-Type adecuado (application/pdf, application/xml) y Cache-Control: no-store, X-Dgii-Demo: 1 en los demo.

6.3 Ejemplos de contrato

```
POST /api/dgii/notas-credito/create
body: { sourceInvoiceId: string, motivo: string, codigoModificacion: 1|2|3|4|5, rncComprador?: string }
200: { ncEcf:"34", ncEncf, sourceEcfNumber, codigoModificacion, motivo, indicadorNotaCredito, unsignedXml, signedXml, signedAt, securityCode, qrUrl, pdfBase64, ambiente:"testecf", mockTrackId, warning }
400: { error: "codigoModificacion invalido. Validos: 1, 2, 3, 4, 5" } // o source 33/34 no permitido
404: { error: "Factura mock '<id>' no encontrada" }

POST /api/dgii/invoices/testecf-send (autenticado, dry-run)
body: { tipoEcf:"31"|"32"|"33"|"34" }
200: { ok:true, mode:"dry-run", preflight:{ prepared:{ xmlUnsigned, xmlSigned, securityCode, qrUrl, ambiente:"testecf", xsdValidation:{status,errors}, signatureDigest }, emisor:{...}, mode
```

```
: "dry-run" } }  
503: { ok:false, reason:"preflight_failed", code:"FEATURE_DISABLED", message:"Fase G deshabilitada." }
```

6.4 Server actions

Accion	Firma	Notas
signIn(formData)	-> AuthResult	Zod email+password<=8; mock o Supabase; detecta MFA
verifyMfa(code)	-> AuthResult	TOTP via Supabase MFA
signOut()	-> void	limpia sesion, redirige /login
uploadCertificateAction(formData)	-> UploadResult	.p12/.pfx <=64KB; parse PKCS#12; cifra AES-256-GCM; persiste; desactiva anterior
loadActiveCertificateAction()	-> PublicCertificate \	null

6.5 Killswitches / feature flags

DATA_SOURCE, isSupabaseConfigured(), isDgiiConfigured(), isCertificateUploadEnabled() (requiere supabase + DGII_CERT_ENCRYPTION_KEY), DGII_TESTECF_SEND_ENABLED (Fase G, hoy bloqueado), DGII_ENVIRONMENT. Además en BD: businesses.dgii_enabled y dgii_settings.dgii_enabled_real_send. Filosofía: TODOS deben coincidir para una acción fiscal real.

7. UI y design system

7.1 Shell de navegacion

Layout: sidebar fijo (w-64) + header sticky (h-16) + contenido fluido.

- Header: badge de empresa + selector de sucursal + direccion | buscador global | notificaciones + avatar de usuario con iniciales y rol.

- Sidebar: logo (icono HeartPulse) + grupos colapsables: General (Dashboard); Clientes; Ventas

(POS, Ventas, Proformas, Pagos, Devoluciones, Notas credito, Caja); Productos; Inventario; Conteo fisico; Reportes; Comunicacion (WhatsApp); IA; Integraciones; DGII (Habilitacion, Configuracion, Secuencias, Facturas, Envios, Certificado); Administracion. Item activo: bg-primary/10 + text-accent. Link "Super Admin" abajo (acento violeta).

7.2 Design system (tokens)

Tailwind 4 via @tailwindcss/postcss (sin tailwind.config.ts). Tokens en globals.css (:root):

Token	Valor	Uso
--brand-primary	#2db4a8	teal principal
--brand-accent	#1a7f8e	teal oscuro (hover/acentos)
--brand-fg	#0f2933	texto principal
--brand-bg	#f7fbfb	fondo
--brand-success	#16a34a	OK / positivo
--brand-warn	#f59e0b	advertencia
--brand-danger	#dc2626	error / destructivo

Tipografia: stack del sistema (ui-sans-serif, system-ui, Segoe UI, Roboto), sin next/font; font-feature-settings: "ss01", "cv11"; antialiased. Sin dark mode. Bordes border-black/5, radios rounded-lg (inputs/botones) y rounded-2xl (cards). Sombras shadow-sm/shadow-lg. Focus ring-2 ring-primary/20; click active:scale-[0.98]. @media print para recibos 80mm. Iconos lucide-react.

7.3 Componentes UI base (custom, en components/ui)

Componente	Variantes / props
Button	variant primary/secondary/outline/ghost/danger; size sm/md/lg/icon
Card (+Header/Title/Content/Footer)	contenedor base
Badge	tone neutral/primary/success/warning/danger/info/purple; outlined
Input/Textarea/Select/Label/HelpText	formularios
Table (+THead/TBody/TR/TH/TD)	tablas de datos
Tabs (+List/Trigger/Content)	basado en contexto
Toast (useToast)	show/success/error
ConfirmDialog	open, title, message, destructive

Componente	Variantes / props
EmptyState	icon, title, description, action
StatCard	label, value, hint, delta, icon, tone
RowActions	viewHref, editHref, onDelete, customActions, can*
PageHeader / SearchInput / FilterBar / SortableTH / ProductImage / BarChart/Spark	utilitarios

Badges por estado: success emerald-50/700, warning amber-50/700, danger rose-50/700, info sky-50/700, primary brand-primary/15 + accent.

7.4 Pantallas del modulo DGII (prioridad)

- /dgii (overview): alert de modulo inactivo; StatCards (aceptados, en proceso, rechazados, secuencias por vencer); badges de tipos e-CF habilitados.
- /dgii/habilitacion (wizard 10 pasos): panel de estado + progreso, "proximo paso recomendado", TestecfReadinessPanel (gaps + boton bloqueado), revision diagnostica por paso, tarjetas de paso expandibles con checklist y selector de estado, paso final auto-calculado, tabla de URLs planificadas, permisos y leyenda de estados.
- /dgii/configuracion: identidad comercial, contacto, ubicacion fiscal (provincia->municipio en cascada), actividad economica, radio de ambiente (testecf/certecf/ecf).
- /dgii/certificado: subir .p12/.pfx + password; tarjeta de estado del certificado.
- /dgii/secuencias: gestion de rangos e-NCF.
- /dgii/certificacion: pruebas pre-certificacion (31-34).
- /dgii/facturas y /dgii/facturas/[id]: listado de e-CF (e-NCF, tipo, cliente, totales, estado, TrackId, acciones) y detalle + Nota de Credito.
- /dgii/preview/[id], /dgii/envios, /dgii/reportes.

Wizard de habilitacion - los 10 pasos

#	Paso	Ruta destino
1	Certificado digital (6 items)	/dgii/certificado
2	Configuracion fiscal (8 items)	/dgii/configuracion
3	Generacion postulacion (7 items)	/dgii/habilitacion
4	Pruebas simulacion e-CF (7 items)	/dgii/certificacion
5	Representaciones impresas (9 items)	/dgii/facturas
6	URLs servicios produccion (7 items)	/dgii/configuracion
7	Declaracion jurada (9 items)	/dgii/habilitacion
8	Autorizacion representante e-CF (9 items)	/dgii/habilitacion
9	Asignacion roles y NCF (9 items)	/admin/permisos
10	Estado final (auto-calculado)	/dgii/habilitacion

7.5 Pantallas POS / Caja

- /pos: terminal 2 columnas. Izq: buscador + grid de productos + lineas de carrito (cantidad,

descuento por linea). Der: selector de cliente, tipo de comprobante (Consumo->32 / Credito Fiscal->31, exige RNC), descuento global, resumen (subtotal, descuento, ITBIS, total), metodo de pago (efectivo/tarjeta/transferencia), monto recibido/vuelto, boton "Emitir proforma". FEFO automatico (lotes vencidos/cuarentena bloqueados).

- /caja: sin sesion (abrir caja) o sesion activa (StatCards de apertura/efectivo/tarjeta/

transferencia; tabla de proformas seleccionables para e-CF; panel de cierre estimado con efectivo contado; sesiones cerradas recientes).

El resto de modulos (clientes, productos, inventario, conteo fisico, reportes, ventas, pagos, proformas) siguen el patron PageHeader + FilterBar + Table + RowActions; varios estan en modo mock/ placeholder (devoluciones, notas-credito, stock por lote, movimientos, cuarentena, recall, whatsapp, ia, varias de admin).

8. Logica de negocio y estados

8.1 ITBIS y totales

```
// por linea
monto_gravado = quantity * unit_price - discount
itbis_linea   = monto_gravado * itbis_rate      // 18 | 16 | 0
total_linea   = monto_gravado + itbis_linea
// por comprobante
subtotal_gravado = SUM(monto_gravado where rate>0)
subtotal_exento  = SUM(monto_gravado where rate=0)
total_itbis      = SUM(itbis_linea)
total = subtotal_gravado + subtotal_exento + total_itbis + total_otros_impuestos
```

8.2 Secuencias e-NCF (reserva atomica)

e-NCF = prefijo(3, ej. E31) + 10 digitos. Reserva con bloqueo de fila:

```
create or replace function public.reserve_ecf_sequence_number(
  p_business_id uuid, p_tipo_ecf text, p_ambiente text) returns int language plpgsql as $$
declare v_seq_id uuid; v_next int; v_range_end int;
begin
  select id, next_number, range_end into v_seq_id, v_next, v_range_end
  from ecf_sequences
  where business_id=p_business_id and tipo_ecf=p_tipo_ecf and ambiente=p_ambiente
    and status='active' and next_number <= range_end
  order by created_at asc limit 1 for update;      -- lock de concurrencia
  if v_seq_id is null then return null; end if;
  update ecf_sequences
    set next_number = v_next + 1,
        status = case when v_next+1 > v_range_end then 'exhausted' else status end,
        updated_at = now()
  where id = v_seq_id;
  return v_next;
end; $$;
```

8.3 Flujo POS -> e-CF

El resolutor (features/sales/document-resolver.ts) decide segun la forma de pago:

- Efectivo / Transferencia -> proforma fiscal (no consume e-NCF); espera el cierre de caja y se convierte a e-CF segun un porcentaje configurable.
- Tarjeta / POS bancario -> e-CF inmediato tipo 31 (consume e-NCF). payment_methods define requires_immediate_ecf y default_ecf_type.

8.4 Cierre de caja con porcentaje

Configurable en dgii_settings: porcentaje por defecto, min/max, si permite cambiarlo, si exige autorizacion de admin bajo 100%, y si auto-genera e-CF al cerrar. El cierre suma por metodo de pago, lista proformas pendientes, aplica el %, selecciona proformas hasta target = total_pendiente * % / 100 y genera los e-CF. Cada cambio de % queda en cash_closing_percentage_logs (auditoria inmutable); el cierre en cash_closings.

8.5 Maquinas de estado

```
Proforma: draft -> issued -> { paid | partially_paid | pending_ecf | pending_cash_closing }
           -> { selected_for_ecf -> ecf_generation_pending -> converted_to_ecf | closed_without_ecf }
           -> { cancelled | expired | voided }

e-CF:      draft -> generated -> validated -> signed -> submitted
           -> in_process -> { accepted | accepted_conditional | rejected }
           -> { cancelled | error | voided }
```

9. Fases, gaps y guía de reconstrucción

9.1 Estado por fase

Fase	Objetivo	Estado
0-1	Diagnostico + base SaaS multi-tenant (auth, RLS)	[OK]
2-3	Modelo de datos DGII/e-CF + UI	[OK]
4-5	Builder XML + validación XSD	[OK]
6-7	Firma digital + Supabase real (0001-0009)	[OK]
8-9	Wizard habilitación + certificado real en preview	[OK]
10-11	Validación local del certificado + checklist pre-Fase G	[OK]
12	Dry-run de Fase G (sin HTTP)	[OK]
13	Postulación + carga de secuencias ante DGII	[EXTERNO cliente]
14 (G)	Envío real a testecf	[BLOQUEADA]
15 (H)	Polling de TrackId / estado	[BLOQUEADA]
16-17	Certificación formal + producción (ecf)	[BLOQUEADA]

9.2 Gaps conocidos (para completar la reconstrucción)

- Autenticación DGII (semilla -> firma -> token) y cache de token: no implementada.
- Cliente HTTP real a testecf/ecf y recepción multipart: stubs (testecf-client.ts).
- Polling de TrackId con reintentos (Fase H): pendiente.
- Persistencia de POS/cierre: parte de los stores son localStorage (mock); falta cablear a BD.
- DgiiSequenceService / settings persistidos: el mapeo demo sintetiza eNCF y usa emisor hardcoded.
- Algoritmo oficial del código de seguridad y formato del QR (D-06): heurística por validar.
- Municipio/Provincia como enum oficial DGII (D-15): hoy omitidos/strings.
- Timezone de fechas (D-03): formateo local sin forzar America/Santo_Domingo.
- Observabilidad/alertas del módulo fiscal y rollback de migraciones: documentar.
- Lint: no hay ESLint/Prettier configurado; packages/ del monorepo está vacío.

9.3 Orden de construcción recomendado

1. Andamiaje: monorepo pnpm + Next.js 15 + Tailwind 4 + componentes UI base + AppShell + DATA_SOURCE=mock.
2. Dominio y tipos: modelos TypeScript (Product, Proforma, e-CF...) + datos mock; navegar todas las pantallas.
3. Base de datos: migraciones 0001-0009 (core/tenancy -> inventario -> clientes -> DGII/POS -> permisos/roles -> helpers auth -> fixes RLS). Aplicar seeds.
4. Auth + RLS: funciones auth_*(), políticas por tabla, middleware, clientes Supabase, sesión/JWT.

Cambiar a DATA_SOURCE=supabase y validar aislamiento por tenant.

- 5. Motor e-CF: builder (orden XSD) -> signer (XMLDSig) -> validator (xmllint-wasm + parches) ->

security-code + qr -> pdf. Cubrir con tests (fixtures por tipo).

- 6. POS y caja: terminal, resolutor de documento, secuencias atomicas, cierre con porcentaje, estados.

• 7. Modulo DGII UI: wizard de habilitacion, configuracion, certificado (upload cifrado), facturas, preview, secuencias, reportes.

- 8. API: route handlers (demo + autenticados) y server actions con sus contratos y killswitches.

- 9. Endurecimiento: permisos por rol, auditoria, observabilidad, CI (typecheck+unit+build+e2e).

- 10. Fase G+ (solo con autorizacion): autenticacion DGII, cliente HTTP, polling TrackId, certificacion.

> Recordatorio final: este blueprint describe el sistema a la fecha de portada. Cualquier emision > fiscal real frente a la DGII requiere autorizacion operativa explicita y la postulacion del > contribuyente. Reconstruir el sistema reproduce la funcionalidad, NO habilita la operacion fiscal.